Operating Systems

Kiarash Sanei

Febuary, 2026

Contents

1 Introduction **2**
- 1.1 General Picture
- 1.2 What is an Operating System?
- 1.3 OS: Mandatory or Optional?
- 1.4 Computer System Structure
- 1.5 Components of a Computer System
- 1.6 The Role of Operating Systems
- 1.7 Operating System Story
- 1.8 Operating System Definition
- 1.9 Computer Startup
- 1.10 Baseboard Management Controller (BMC)
- 1.11 Computer System Organization
- 1.12 I/O & Device Driver
- 1.13 Storage Structure
- 1.14 Caching
- 1.15 Storage-Device Hierarchy
- 1.16 Direct Memory Access Structure
- 1.17 Computer-System Architecture
- 1.18 A Dual-Core Design
- 1.19 Operating System Structure
- 1.20 Multiprogramming vs. Multitasking vs. Multiprocessing
- 1.21 Memory Layout for Multiprogrammed System
- 1.22 Operating System Operations
- 1.23 Transition from User to Kernel Mode

2 Operating System Structures **12**
- 2.1 Operating System Services
- 2.2 CLI
- 2.3 GUI
- 2.4 Touchscreen Interfaces

2.5 System Calls . . . .
2.6 System Call Implementation . . . .
2.7 System Call Parameter Passing . . . .
2.8 Types of System Calls . . . .
2.9 System Programs . . . .
2.10 OS Design and Implementation . . . .
2.11 Implementation . . . .
2.12 Operating System Structure . . . .
2.13 Simple Structure: MS-DOS . . . .
2.14 More complex: UNIX . . . .
2.15 Layered Approach . . . .
2.16 Microkernel System Structure . . . .
2.17 Modules . . . .
2.18 Solaris Modular Approach . . . .
2.19 Hybrid Systems . . . .
2.20 Virtual Machines . . . .
2.21 Docker/Container vs. Virtualization . . . .

3 Processes **22**
3.1 Process Concept . . . .
3.2 Process in Memory . . . .
3.3 Process State . . . .
3.4 Process Control Block (PCB) . . . .
3.5 Threads . . . .
3.6 Process Representation in Linux . . . .
3.7 Process Scheduling . . . .
3.8 Schedulers . . . .
3.9 Medium Term Scheduling . . . .
3.10 Context Switch . . . .
3.11 Operations on Processes . . . .
3.12 Process Creation . . . .
3.13 Process Termination . . . .
3.14 Multiprocess Architecture – Chrome Browser . . . .
3.15 Interprocess Communication . . . .
3.16 Shared-Memory Solution: Producer-Consumer Problem . . . .
3.17 Shared Memory . . . .
3.18 Message Passing . . . .
3.19 Communication Link . . . .
3.20 Direct Communication . . . .
3.21 Indirect Communication . . . .

3.22 Synchronization . . . .
3.23 Buffering . . . .
3.24 Linux Pipes . . . .

4 Threads **31**
4.1 Multithreaded Server Architecture . . . .
4.2 Single and Multithreaded Processes . . . .
4.3 Multi-Threading in OS Kernels . . . .
4.4 Benefits of Multi-Threading . . . .
4.5 Threads vs. Processes . . . .
4.6 Multicore Programming . . . .
4.7 Concurrency vs. Parallelism . . . .
4.8 Amdahl's Law . . . .
4.9 User Threads and Kernel Threads . . . .
4.10 Multithreading Models . . . .
4.11 Many-to-One . . . .
4.12 One-to-One . . . .
4.13 Many-to-Many . . . .
4.14 Two-level Model . . . .
4.15 Processes, User Threads, Kernel Threads . . . .
4.16 Trade-Offs . . . .
4.17 Thread Libraries . . . .
4.18 Pthreads . . . .
4.19 Java Threads . . . .
4.20 Implicit Threading . . . .
4.21 Threading Issues . . . .
4.22 Thread Pools . . . .
4.23 Scheduler Activations . . . .

5 CPU Scheduling **38**
5.1 Basic Concepts . . . .
5.2 CPU Scheduler . . . .
5.3 Dispatcher . . . .
5.4 Scheduling Criteria . . . .
5.5 Scheduling Algorithm Optimization Criteria . . . .
5.6 First Come, First Served (FCFS) Scheduling . . . .
5.7 Shortest Job First (SJF) Scheduling . . . .
5.8 Determining Length of Next CPU Burst . . . .
5.9 Shortest Remaining Time First . . . .
5.10 Priority Scheduling . . . .
5.11 Round Robin (RR) . . . .

5.12 Multilevel Queue . . . .
5.13 Multilevel Feedback Queue . . . .
5.14 Thread Scheduling . . . .
5.15 Pthread Scheduling . . . .
5.16 Multiple-Processor Scheduling . . . .
5.17 NUMA and CPU Scheduling . . . .
5.18 Multiple-Processor Scheduling – Load Balancing . . . .
5.19 Multicore Processors . . . .
5.20 Conclusion: Levels of Scheduling . . . .
5.21 Real-Time CPU Scheduling . . . .
5.22 Features of Real-Time Kernels . . . .
5.23 Implementing RT OS . . . .
5.24 Priority-based Scheduling . . . .
5.25 Rate Monotonic Scheduling . . . .
5.26 Earliest Deadline First Scheduling (EDF) . . . .
5.27 POSIX Real-Time Scheduling . . . .

6 Memory **45**

6.1 Base and Limit Registers . . . .
6.2 Address Binding . . . .
6.3 Binding of Instructions and Data to Memory . . . .
6.4 Multistep Processing of a User Program . . . .
6.5 Logical vs. Physical Address Space . . . .
6.6 Dynamic Relocation using a Relocation Register . . . .
6.7 Dynamic Linking . . . .
6.8 Swapping . . . .
6.9 Context Switch Time and Swapping . . . .
6.10 Memory Management Schemes . . . .
6.11 Contiguous Allocation . . . .
6.12 HW Support for Relocation and Limit Registers . . . .
6.13 Multiple-Partition Allocation . . . .
6.14 Dynamic Storage-Allocation Problem . . . .
6.15 Fragmentation . . . .
6.16 Segmentation . . . .
6.17 Segmentation Architecture . . . .
6.18 Segmentation vs. Variable-Sized Contiguous Allocation . . . .
6.19 Problems with Segmentation . . . .
6.20 Paging . . . .
6.21 Address Translation Scheme . . . .
6.22 Implementation of Page Table . . . .

6.23 Associative Memory . . . .
6.24 Effective Access Time . . . .
6.25 Memory Protection . . . .
6.26 Shared Pages . . . .
6.27 Structure of the Page Table . . . .
6.28 Hierarchical Page Tables . . . .
6.29 Hashed Page Tables . . . .
6.30 Inverted Page Table . . . .

7 Virtual Memory **55**
7.1 Virtual-Address Space . . . .
7.2 Demand Paging . . . .
7.3 Basic Concepts . . . .
7.4 Valid-Invalid Bit . . . .
7.5 Page Fault . . . .
7.6 Aspects of Demand Paging . . . .
7.7 Instruction Restart . . . .
7.8 Performance of Demand Paging . . . .
7.9 Copy-on-Write . . . .
7.10 What Happens if There is no Free Frame? . . . .
7.11 Page Replacement . . . .
7.12 Basic Page Replacement . . . .
7.13 Page and Frame Replacement Algorithms . . . .
7.14 First-In-First-Out (FIFO) Algorithm . . . .
7.15 Optimal Algorithm . . . .
7.16 Least Recently Used (LRU) Algorithm . . . .
7.17 LRU Approximation Algorithms . . . .
7.18 Enhanced Second-Chance Algorithm . . . .
7.19 Counting Algorithms . . . .
7.20 Page-Buffering Algorithms: Enhanced Techniques . . . .
7.21 Applications and Page Replacement . . . .
7.22 Allocation of Frames . . . .
7.23 Fixed Allocation . . . .
7.24 Priority Allocation . . . .
7.25 Global vs. Local Allocation . . . .
7.26 Non-Uniform Memory Access . . . .
7.27 Thrashing . . . .
7.28 Demand Paging and Thrashing . . . .
7.29 Working-Set Model . . . .
7.30 Keeping Track of Working Set . . . .

7.31 Page-Fault Frequency . . . .
7.32 Working Sets and Page Fault Rates . . . .
7.33 Memory-Mapped Files . . . .
7.34 Memory-Mapped File Technique for all I/O . . . .
7.35 Allocating Kernel Memory . . . .
7.36 Buddy System . . . .
7.37 Slab Allocator . . . .

8 Process Synchronization **66**
8.1 Critical Section Problem . . . .
8.2 Solution to Critical-Section Problem . . . .
8.3 Race Conditions in OS Kernel . . . .
8.4 Critical-Section Handling in OS . . . .
8.5 Preemptive vs. Non-Preemptive Kernels . . . .
8.6 Peterson's Solution . . . .
8.7 Synchronization Hardware . . . .
8.8 Solution to CS Problem using Locks . . . .
8.9 test_and_set Instruction . . . .
8.10 compare_and_swap Instruction . . . .
8.11 Bounded-waiting Mutual Exclusion with test_and_set . . . .
8.12 Mutex Locks . . . .
8.13 `acquire()` and `release()` . . . .
8.14 Semaphore . . . .
8.15 Semaphore Usage . . . .
8.16 Semaphore Implementation . . . .
8.17 Semaphore Implementation with no Busy waiting . . . .
8.18 Deadlock and Starvation . . . .
8.19 Classical Problems of Synchronization . . . .
8.20 Problems with Semaphores . . . .
8.21 Monitors . . . .
8.22 Condition Variables . . . .
8.23 Monitor Solution to Dining Philosophers . . . .

9 Deadlocks **76**
9.1 System Model . . . .
9.2 Deadlock Characterization . . . .
9.3 Resource-Allocation Graph . . . .
9.4 Basic Facts . . . .
9.5 Methods for Handling Deadlocks . . . .
9.6 Deadlock Prevention . . . .
9.7 Deadlock Example with Lock Ordering . . . .

9.8 Deadlock Avoidance . . . .
9.9 Safe State . . . .
9.10 Basic Facts . . . .
9.11 Avoidance Algorithms . . . .
9.12 Resource-Allocation Graph Scheme . . . .
9.13 Resource-Allocation Graph Algorithm . . . .
9.14 Deadlock Detection . . . .
9.15 Single Instance of Each Resource Type . . . .
9.16 Resource-Allocation Graph and Wait-for Graph . . . .
9.17 Detection-Algorithm Usage . . . .
9.18 Recovery from Deadlock: Process Termination . . . .
9.19 Recovery from Deadlock: Resource Preemption . . . .

10 File System **82**
10.1 File Concept . . . .
10.2 File Attributes . . . .
10.3 File Operations . . . .
10.4 Open Files . . . .
10.5 Open File Locking . . . .
10.6 File Locking Example: Java API . . . .
10.7 File Types: Name, Extension . . . .
10.8 File Structure . . . .
10.9 Access Methods . . . .
10.10Simulation of Sequential Access on Direct Access File . . . .
10.11Unix/POSIX Idea: Everything is a File . . . .
10.12Connecting Processes, File Systems, and Users . . . .
10.13I/O and Storage Layers . . . .
10.14C High-Level File API: Streams . . . .
10.15C API Standard Streams: `stdio.h` . . . .
10.16C High-Level File API . . . .
10.17C Streams: Char-by-Char I/O . . . .
10.18C Streams: Block-by-Block I/O . . . .
10.19C High-Level File API: Positioning Pointer . . . .
10.20Low-Level File I/O: RAW system-call interface . . . .
10.21Filesystem . . . .
10.22Building a File System . . . .
10.23User vs. System View of a File . . . .
10.24Disk Management . . . .
10.25Disk Structure . . . .
10.26What Does File System Need? . . . .

10.27Types of File Systems . . . .
10.28Critical Factors in File System Design . . . .
10.29Operations Performed on Directory . . . .
10.30Directory Organization . . . .
10.31Single-Level Directory . . . .
10.32Two-Level Directory . . . .
10.33Tree-Structured Directories . . . .
10.34Acyclic-Graph Directories . . . .
10.35General Graph Directory . . . .
10.36File System Mounting . . . .
10.37File-System Structure . . . .
10.38File System Layers . . . .
10.39File-System Implementation . . . .
10.40In-Memory File System Structures . . . .
10.41Partitions and Mounting . . . .
10.42Virtual File Systems . . . .
10.43Virtual File System Implementation . . . .
10.44Directory Implementation . . . .
10.45Allocation Methods: Contiguous . . . .
10.46Extent-Based Systems . . . .
10.47Allocation Methods: Linked . . . .
10.48Linked Allocation . . . .
10.49Allocation Methods: Indexed . . . .
10.50Performance . . . .
10.51Free-Space Management . . . .
10.52Linked Free Space List on Disk . . . .
10.53Efficiency and Performance . . . .
10.54Page Cache . . . .
10.55Unified Buffer Cache . . . .
10.56Recovery . . . .
10.57Log Structured File Systems . . . .

11 Mass-Storage Systems **99**

11.1 Overview of Mass Storage Structure . . . .
11.2 Hard Disk Performance . . . .
11.3 Solid-State Disks . . . .
11.4 Magnetic Tape . . . .
11.5 Disk Structure . . . .
11.6 DAS vs. NAS vs. SAN . . . .
11.7 Disk Attachment . . . .

11.8 Storage Array . . . . . .
11.9 Storage Area Network . . . . . .
11.10Network-Attached Storage . . . . . .
11.11Why to use SAN instead of Local Drives? . . . . . .
11.12SSDs: Quick Introduction . . . . . .
11.13Applications and Data Accesses . . . . . .
11.14Hybrid Storages: Tiering . . . . . .
11.15Disk Scheduling . . . . . .
11.16SSTF . . . . . .
11.17SCAN . . . . . .
11.18C-SCAN . . . . . .
11.19C-LOOK . . . . . .
11.20Selecting a Disk-Scheduling Algorithm . . . . . .
11.21Disk Management . . . . . .
11.22Swap-Space Management . . . . . .
11.23RAID Structure . . . . . .
11.24Other Features . . . . . .

1 Introduction

1.1 General Picture

- Computer Abstraction Levels
    1. Applications
    2. Compilers
    3. Operating System: Almost Nearest SW Layer to ISA
    4. Computer Arch. (ISA)
    5. Microarchitecture
    6. Digital Design
    7. Circuit
    8. Device

1.2 What is an Operating System?

- A Program that Acts as an Intermediary between a User of a Computer and Computer HW
- Acts as a governor
- Executes user programs and makes solving user problems easier
- Allows multiple programs to simultaneously be executed in a shared HW
- Uses computer HW in an efficient manner
- Makes computer system convenient to use

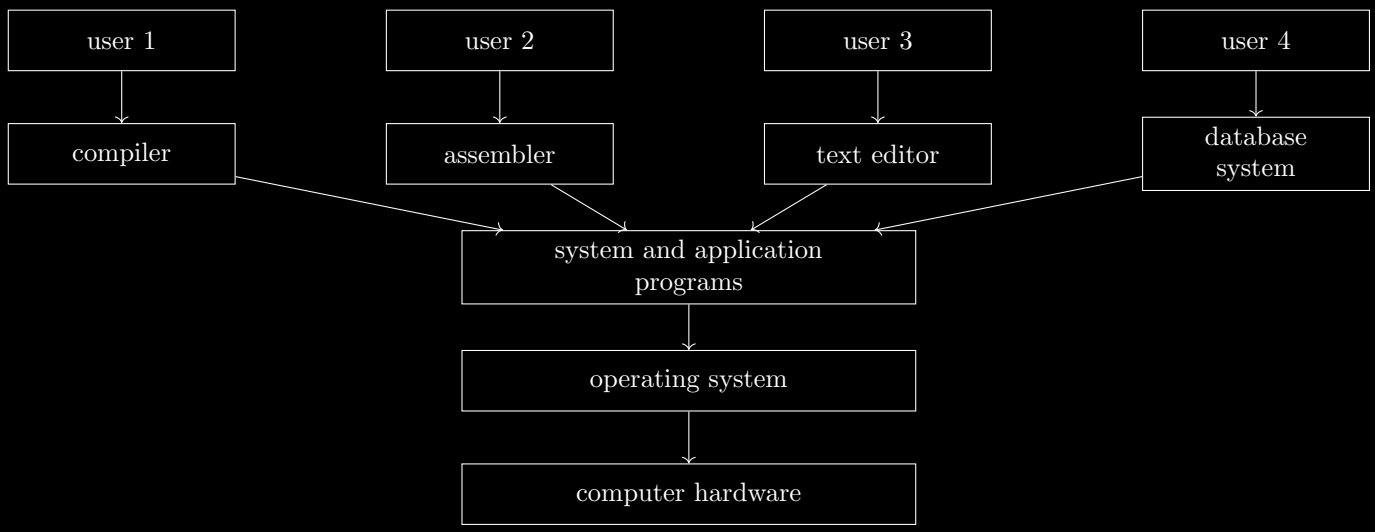
1.3 OS: Mandatory or Optional?

- Can a computer run without an operating system?
    - Yes, earliest computers didn't have any OS
- What does a computer without an OS look like?
    - Machines tasked with one program at a time
        - Cannot look at a doc and keep a clock running on your desktop
    - Each program has a lot of work to do
        - Where to load a program
        - IO access

1.4 Computer System Structure

- HW
    - Provides basic computing resources
    - CPU, memory, storage, and I/O devices
- OS
    - Controls and coordinates use of HW among various applications and users
- Application & System Programs
    - Define ways in which system resources used to solve computing problems of users
    - Word processors, compilers, web browsers, database systems, video games
- Users
    - People, machines, other computers

1.5 Components of a Computer System



Computer System Architecture

-

1.6 The Role of Operating Systems

- Major parameters in which we would like to optimize OS towards to
- We need to take a look at the role of OS from two perspectives
- Depends on the Point of View
  - Users view
  - System view
- Users want Convenience, Ease of Use
  - Throughput not important
  - Latency is, however, of importance
  - Don't care about resource utilization
- Shared Computer such as Mainframe or Minicomputer must keep all users happy
  - Resource utilization
  - Fair share between all users
- Also Depends on Type of System Usage
  - Embedded, desktop, high-performance computing
    - Objectives: power, performance, ease of use, utilization, real-time, reliability, etc
  - Users of dedicate systems such as workstations have dedicated resources but frequently use shared resources from servers
    - Compromise between individual usability & resource utilization
  - Handheld computers are resource poor, optimized for usability and battery life
  - Some have Little or No User Interface
    - E.g. embedded computers in devices & automobiles
- OS is a Resource Allocator
  - Possible resources include CPU time, memory space, file-storage space, I/O devices, etc.
  - Manages all resources

    - How, when, and for how long allocate resources to programs and users
    - Decides between conflicting requests for efficient and fair resource use
- OS is a Control Program
    - Controls execution of programs to prevent errors and improper use of computer

1.7 Operating System Story

- Fundamental Goal of a Computer System
    - Execute user programs and to make solving user problems easier
- Computer HW Constructed
    - HW alone not easy to use
    - Application programs developed
    - Application programs require certain common operations such as those controlling I/O devices
- Common functions of controlling and allocating resources brought together into one piece and called OS

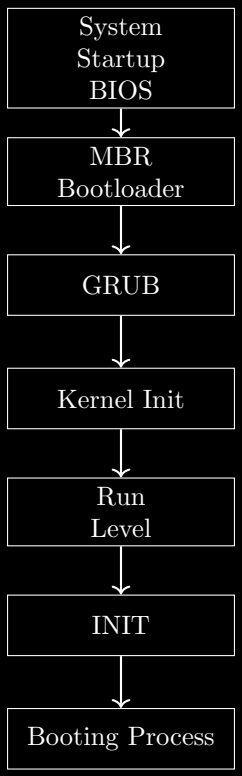
1.8 Operating System Definition

- No Universally Accepted Definition
- "Everything a Vendor Ships when you Order an OS" is a Good Approximation
- "The one Program Running at All Times on the computer" is Kernel
- Everything else is either:
    - A system program (ships with OS)
        - File management programs, loader, linker, compiler
    - An application program

1.9 Computer Startup

- Can we think of any SW layer lower than OS?
- OS resides in disk. How it can be executed?
- Remember stored program concept
    - John von Neumann
    - Both instructions (programs) and data are stored in the same read-write memory (RAM).
    - The memory is addressable, meaning the CPU can fetch either an instruction or a piece of data by its memory address.
    - Instructions are executed sequentially by the CPU, which fetches them from memory.
- Bootstrap Program
    - Loaded at Power-Up or Reboot
    - Typically stored in ROM or EPROM, generally known as firmware
    - Initializes all aspects of a system such as CPU registers, device controllers
    - Loads OS kernel and starts execution
    - OS then starts executing first process

- Bootstrap Program (Linux)



Linux Booting Process

-

1.10 Baseboard Management Controller (BMC)

- BMC Role
  - Small computer that sits on every server MB
  - Typically ARM-based SoC with GUI and built-in logic

1.11 Computer System Organization

- Computer-System Operation
  - One or more CPUs, device controllers connect through common bus providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles
- Each Device Controller is in Charge of a Particular Device Type (e.g., disk drives, audio devices)
- Each Device Controller has a Local Buffer
- I/O: Device $\leftrightarrow$ Local Buffer of Controller
- Device Controller Informs CPU
  - That it has finished its operation by causing an interrupt
- CPU Moves Data from Local buffers to Main memory

1.12 I/O & Device Driver

- Device Driver
  - Alleviates complexity of I/O devices from Kernel
  - For each device controller to manage I/O

    - Provides uniform interface (controller ↔ kernel)
    - Device ↔ Device controller (local buffer) ↔ Device driver
    - Device driver is informed by an interrupt
    - Device driver ↔ Kernel
    - Interrupt Transfers Control to Interrupt Service Routine (Generally, through interrupt vector, which contains addresses of all service routines)
    - Must Save Address of Interrupted Instruction
    - OS Preserves State of CPU by Storing Registers and Program Counter (PC)
    - Determines which Type of Interrupt has Occurred
    - Different Segments of Code Determine what Action Should be Taken for Each Type of Interrupt
    -
- Device Driver vs. Device Controller
    - DC is on the device (HW)
    - DC is like the brain of the device, it literally controls it
    - DC has data buffer
    - DC can translate the high level commands into device-specific aactions
    - DC has status registers
    - DD is a part of OS (SW)
    - There is a DD for every DC
    - DD translates the commands of kernel for DC
    - The OS doesn't have to know how to talk to every DC, it use DD interface
    - DD writes commands into command registers of DC
    - DD has ISR and handels interrupts raised by DC and inform kernel
- A Trap or Exception
    - A SW-generated interrupt caused either by an error or a User Request (aka, system call)

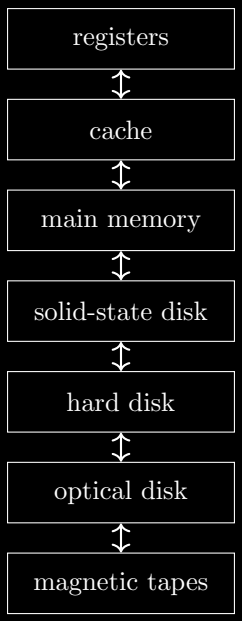
1.13 Storage Structure

- Main Memory
    - Only large storage media that CPU can access directly
    - Random access and typically volatile
    - Exception: Intel Optane DIMMs (also called, NVDIMMs)
- Secondary Storage
    - Extension of main memory that provides large nonvolatile storage capacity
    - Hard Disk Drives (HDDs)
    - Solid-State Drives (SSD) faster then HDD
- Tertiary Storage
    - Optical disk
    - Magnetic tape
- Storage Systems Organized in Hierarchy
    - Ideal storage is inexpensive, fast, and non-volatile
    - Ranked based on Speed, cost, and volatility
- Caching
    - Information in use copied from slower to faster storage temporarily
    - Performed at various levels: HW, OS, and application

1.14 Caching

- Faster Storage (cache) Checked First
- To determine if information is there
- If it is, information used directly from cache (fast)
- If not, data copied to cache and used there
- Cache is Smaller than Storage being Cached, so there is issues
    - Cache management important design problem
    - Cache size and replacement policy

1.15 Storage-Device Hierarchy



Memory Hierarchy

-

1.16 Direct Memory Access Structure

- Used for High-Speed I/O Devices, so they are able to transmit info at close to memory speeds
- Device Controller Transfers Data Blocks from buffer storage directly to main memory without CPU intervention
- Only one Interrupt is generated per Block rather than one interrupt per byte

1.17 Computer-System Architecture

- Single General-Purpose Processor
    - Commonly used in embedded and thin clients
    - Most systems have special-purpose processors as well
- Multiprocessors Systems
    - Growing in use and importance
    - Mainly used in server-class systems
    - Aka parallel systems, tightly-coupled systems
    - Pros
        - Increased throughput

        - Economy of scale
        - Increased reliability – graceful degradation or fault tolerance
    - Asymmetric Multiprocessing (AMP)
        - Each processor is assigned a specific task
        - Master/slave model
        - Master: schedules and allocates work to slave processors
    - Symmetric Multiprocessing (SMP)
        - Each processor performs all tasks
        - All processors are peers
        - No Master-slave relationship
    - N CPUs with N concurrent processes
    - Share physical memory

1.18 A Dual-Core Design

- Multi-chip vs. Multicore
- Early Trend: Multi-Chip Processors
- Current Trend: Multicore
    - Faster communications between cores
    - More energy efficiency
    - No difference from OS perspective

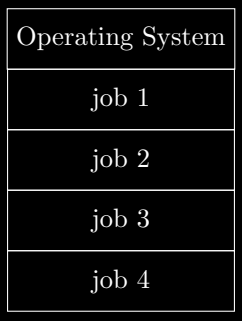
1.19 Operating System Structure

- Multiprogramming (Batch system)
    - Aimed at improving system efficiency
    - Single user cannot keep CPU and I/O devices busy at all times
    - Multiprogramming organizes jobs (code and data) so CPU always has one to execute
    - Job pool
        - All jobs waiting to be executed reside either in disk or memory
        - A subset of total jobs in system is kept in memory
    - When it has to wait (e.g. for I/O), OS switches to another job
        - In Non-multiprogrammed system, CPU sits idle.
- Multiprogrammed System
    - Provides environment to effectively utilize resources
    - Does not provide user interaction
- Timesharing (Multitasking)
    - Logical extension of multi-programmed
    - CPU switches jobs so frequently that users can interact with each job, creating interactive computing
    - Response time should be less than 1 second
    - Each user is given impression that entire computer system is dedicated to his/her use
    - Process: A program loaded into memory and executing
    - A process executes for only a short time before it either finishes or needs to perform I/O
    - Time sharing and multiprogramming requires several jobs be kept in main memory
    - Job scheduling: Decides which job to be brought to memory
    - CPU scheduling: Decides which job to be executed
    - If processes don't fit in memory, swapping moves them in and out to run
    - Virtual memory allows execution of processes not completely in memory. It separates logical memory (as viewed by users) from physical memory

1.20 Multiprogramming vs. Multitasking vs. Multiprocessing

- Single Processing Non-Multi-programmed
  - Processes are done one by one in a series. When a process ends, other one starts.
- Multiprocessing
  - Processes are done parallely.
- Single Processing Multi-Programming
  - Processes are done in a series. When a process ends or reaches I/O, other one starts.
- Single Processing Multi-Tasking
  - Processes are done in a series. When a process's time ends, other one starts.
- Multi-Processing Multi-Tasking
  - Processes are done parallely. In each line, when a process's time ends, other one starts.

1.21 Memory Layout for Multiprogrammed System



Operating System Job Management

-

1.22 Operating System Operations

- Modern OS are Interrupt Driven
  - No process to execute so no I/O devices to service and no users to whom to respond so OS will sit idle and wait for something to happen, i.e., interrupt
  - Events are signaled by occurrence of an interrupt
- Interrupts
  - Hardware Interrupt
    - Hardware Interrupt by One of Devices
    - Disk drive, NIC, printer, and others
  - Software Interrupt (Exception or Trap)
    - Software error
      - Division by zero
      - Invalid memory access
      - Access to kernel address space
    - Request for OS service from a user program
    - Other process problems
      - Infinite loop
      - Processes modifying each other or OS
- OS Needs to Manage All System Resources

    - It needs some kind of privileged instructions only executable by OS
- How Privileged vs. Non-Privileged Instructions could be distinguished by CPU?
    - HW Technique
    - SW Technique
- Dual-Mode Operation Allows OS to Protect itself and Other System Components
    - User mode and kernel mode
- Mode Bit
    - Provided by HW: kernel (0) or user (1)
    - Ability to distinguish when code running in user or kernel
- System call
    - Changes mode to kernel, return from call resets to user
    - At system boot time: HW starts in kernel mode
- 3 Types of events to change from user to kernel
    1. System call
    2. Exception
    3. Interrupt
- Main Aim of Dual-Mode Operation
    - Protecting OS from errant users
    - Some instructions may cause harm → designate such instructions as privileged instructions
    - Privileged instructions cannot be executed in user mode
    - Examples of privileged instructions
        - Switching to kernel mode
        - I/O control
        - Setting the timer
        - Interrupt management
        - Clearing memory
        - Remove process from memory
- Flow of a System Call
    1. Treated as a SW interrupt
    2. Control passes through interrupt vector to a service routing in the OS
    3. Mode bit = 0 (kernel mode)
    4. Kernel examines interrupting instruction to determine type of interrupt
    5. Additional information may be found in RF or stack
    6. Kernel verifies passed parameters
    7. System call is executed
    8. Control passed to instruction following system call
    9. Mode bit =1 (user mode)
- Lack of HW-Supported Mode
    - MS-DOS written for 8088 both has no mode bit and no dual mode
- Possible Scenarios in Absence of Dual-Mode
    - User program can wipe out OS
    - Multiple programs can write to a device at same time
- Recent CPUs and OSs Support Dual-Mode
    - Linux, Unix, Solaris, Win-XP, Win-Vista
- Increasingly CPUs Support Multi-Mode Operations
    - Virtual Machine Manager (VMM) mode for guest VMs

1.23 Transition from User to Kernel Mode

- Timer to Prevent Infinite Loop / Process Hogging Resources
  - Timer is set to interrupt computer after some time period
  - Keep a counter that is decremented by physical clock
  - OS sets counter (privileged instruction)
  - Counter = 0 $\rightarrow$ Generates an interrupt
  - Set up timer before scheduling process to regain control or terminate program that exceeds allotted time

2 Operating System Structures

2.1 Operating System Services

- OSs Provide an Environment for Execution of Programs and Services to Programs and Users
- Set of OS Services Provides Functions that are Helpful to Users for Convenience of Programmers
  - User interface
  - Program execution
  - I/O operations
  - File-system manipulation
  - Communications
  - Error detection
- User Interface
  - Almost all OSes have a user interface (UI)
    - Command-Line (CLI)
    - Graphics User Interface (GUI)
    - Batch interface: input commands from file
- Program Execution
  - System must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
- I/O Operations
  - A running program may require I/O, which may involve a file or an I/O device
  - Users cannot directly control I/O devices for sake of protection and efficiency
- File-System Manipulation
  - Programs need to read and write files/dir
  - Create and delete files/dir
  - Search files/dir
  - List file information
  - Permission management
- Communications
  - Processes may exchange information on same computer or between computers over a network
  - Communications may be via shared memory or through message passing (packets moved by OS)
- Error Detection
  - OS needs to be constantly aware of possible errors
  - May occur in CPU and memory HW, in I/O devices, in user program
  - For each type of error, OS should take appropriate action to ensure correct and consistent computing
  - Debugging facilities can greatly enhance user's and programmer's abilities to efficiently use system
- Another Set of OS Functions exist for ensuring Efficient Operation of System itself via Resource Sharing
  - Resource allocation
    - When multiple users/jobs running concurrently, resources must be allocated to each of them
    - Many types of resources:
      - CPU cycles
      - Main memory
      - File storage

        - I/O devices
    - Accounting
        - To keep track of which users use how much and what kinds of computer resources
        - Accounting info can be used for billing or just statistics
        - Such statistics info helps researchers/admins to reconfigure system to improve computing efficiency and services
    - Protection and security
        - Owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
        - Protection
            - Involves ensuring that all accesses to system resources are controlled
        - Security
            - Security of system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

2.2 CLI

- CLI or Command Interpreter
    - Allows direct command entry
    - Sometimes implemented in kernel, sometimes by systems program
    - Sometimes multiple flavors implemented: shells
    - Primarily fetches a command from user and executes it
- Two Approaches in Implementing Shells
    1. Commands built-in
        - For each command, there is a relevant code in Shell
    2. Just names of programs
        - Used in UNIX
        - Does not understand input command
        - Adding new features doesn't require shell modification

2.3 GUI

- User-Friendly Desktop Metaphor Interface
    - Usually mouse, keyboard, and monitor
    - Icons represent files, programs, actions, etc
    - Various mouse buttons over objects in interface cause various actions (provide information, options, execute function, open directory (known as a folder)
- Many Systems now Include both CLI and GUI
    - Microsoft Windows is GUI with CLI "command" shell
    - Apple Mac OS X is "Aqua" GUI interface with UNIX kernel underneath and shells available
    - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

2.4 Touchscreen Interfaces

- Touchscreen Devices Require New Interfaces
    - Mouse not possible or not desired
    - Actions and selection based on gestures
    - Virtual keyboard for text entry
    - Voice commands

2.5 System Calls

- Programming Interface to Services Provided by OS
- Typically Written in a High-Level Language (C/C++)
- Some low-level and most-frequent tasks written in Assembly
- Mostly Accessed by Programs via API rather than direct system call use
- Application Programming Interface (API)
    - Specifies a set of functions that are available to an application programmer
    - Parameters passed to each function
    - Return values programmer can expect
    - Three Most Common APIs
        - Win32 API for Windows
        - POSIX API for POSIX-based systems
            - Including virtually all versions of UNIX, Linux, and Mac OS X
        - Java API for Java Virtual Machine (JVM)
    - APIs Invoke Actual System Calls
        - On behalf of application programmer
- Why to Use APIs?
    - Portability
        - Programmer can expect his/her program to be compiled on any system that supports the same API
        - Some system calls are very detailed and hard to work
    - More coarse-grained component in programming
- Mostly a One-to-One Mapping between APIs and system calls

2.6 System Call Implementation

- System Call Interface
    - Invokes intended system call in OS Kernel
    - Then returns status of system call and any return values
- Caller Needs Know Nothing about how system call is Implemented
- Just needs to obey API and understand what OS will do as a result call
- Most details of OS interface hidden from programmer by API

2.7 System Call Parameter Passing

- Often, More Information Required than Simply Identity of Desired System Call
    - Exact type and amount of information vary according to OS and call
- Three Methods Used to Pass Parameters to OS
    1. Simplest: pass parameters in registers
        - In some cases, may be more parameters than registers
    2. Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
        - This approach taken by Linux and Solaris
    3. Parameters placed, or pushed, onto stack by program and popped off the stack by OS
        - Block and stack methods do not limit number or length of parameters being passed

2.8 Types of System Calls

- Process Control
  - End, abort
  - Create process, terminate process
  - Load, execute
  - Get process attributes, set process attributes
  - Wait for time
  - Wait event, signal event
  - Allocate and free memory
  - Dump memory if error
  - Debugger for determining bugs, single step execution
  - Locks for managing access to shared data between processes or release lock
- File Management
  - Create file, delete file
  - Open, close file
  - Read, write, reposition
  - Get and set file attributes
  - Copy or move a file
- Device Management
  - Request device, release device
  - Read, write, reposition
  - Get device attributes, set device attributes
  - Logically attach or detach devices
  - Various Resources Controlled by OS can be Thought as Devices
    - Physical devices: main memory, disk drives
    - Virtual devices: files
  - Many OSes such as UNIX do not distinguish between physical and virtual devices and use a set of similar system calls
- Communications
  - Create, delete communication connection
  - Send, receive messages (message passing model)
    - Message passing more suitable for small requests
    - Shared-memory model
      - Create and gain access to memory regions
      - Form of data in shared memory: under process control
      - Maximum speed and convenience of communication
- Information Maintenance
  - Get time or date, set time or date
  - Get system data, set system data
  - Number of current users
  - OS version
  - Get and set process, file, or device attributes
  - Debugging info
- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

2.9 System Programs

- Provide a Convenient Environment for Program Development and Execution
- Most Users' View of OS defined by System Programs, not Actual System Calls
- Some are Simply User Interfaces to System Calls others are considerably more complex
- Privileged Instruction → System Call → API → System Program
- Can be Divided into:
  - File manipulation
    - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
  - Status information
    - Some ask system for info: date, time, amount of available memory, disk space, number of users
    - Others provide detailed performance, logging, and debugging information
    - Typically, these programs format and print output to terminal or other output devices
    - Some systems implement a registry that is used to store and retrieve configuration information
  - File modification
    - Text editors to create and modify files
    - Special commands to search contents of files or perform transformations of the text
  - Programming language support
    - Compilers
    - Assemblers
    - Debuggers and interpreters
  - Program loading and execution
    - Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
  - Communications
    - Provide mechanism for creating virtual connections among processes, users, and computer systems
    - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another
  - Background services
    - Launch at boot time
      - Some for system startup, then terminate
      - Some from system boot to shutdown
    - Provide facilities like disk checking, process scheduling, error logging, printing
    - Known as services, subsystems, daemons
  - Application programs
    - web browsers, spreadsheets, database systems, games
    - Run by users
    - Not typically considered part of OS

2.10 OS Design and Implementation

- Internal structure of different OSs can vary widely
- Start design by defining goals and specifications
  - Throughput, energy efficiency, response time, realtime, user-friendly
- Also affected by:
  - Choice of HW
  - Type of system: single user, multiuser, real-time, distributed, general purpose

- User goals vs. System goals
  - User goals: OS should be convenient to use, easy to learn, reliable, safe, and fast
  - System goals: OS should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient
- Policy vs. Mechanism
  - Policies: decide what will be done
  - Mechanism: determine how to do sth
  - Separation of Policy from Mechanism allows maximum flexibility if policy decisions are to be changed later

2.11 Implementation

- Much Variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Usually a Mix of Languages
  - Lowest levels in assembly while main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- High-Level Languages
  - Faster code development
  - Compact
  - Easy to understand and debug
  - Improvements in compiler further optimizes code
  - Easier to Port to other HW
  - Slower
  - Increased storage/memory
- Assembly Languages
  - Faster
  - Only can be run on target ISA
- Hybrid Solution
  - OS is large
  - But small fraction of code is performance bottleneck
- Critical Sections of OS
  - CPU scheduler → Assembly
  - Memory manager → Assembly
  - Other Parts → C/C++

2.12 Operating System Structure

- General-Purpose OS is Very Large Program
- Various Ways to Structure Ones
  - Simple structure: MS-DOS
  - More complex: UNIX
  - Layered: an abstraction
  - Microkernel: Mach

2.13 Simple Structure: MS-DOS

- Originally designed and implemented by very few people
- Written to Provide Most Functionality in Least space
- Not Well-Defined Structure (Not divided into modules)
- Interfaces and Levels of Functionality Not Well Separated (Although it has some structure)
- Very Vulnerable to Errant or Malicious Programs

2.14 More complex: UNIX

- Limited by HW Functionality
  - Original UNIX OS had limited structuring
- Consists of Two Separable Parts
  - Systems programs
  - Monolithic Kernel
    - Consists of everything below system call interface and above physical HW
    - Provides file system, CPU scheduling, memory management, and other OS functions
    - A large number of functions for one level

2.15 Layered Approach

- OS Divided into a Number of Layers (Levels)
  - Each built on top of lower layers
  - Bottom layer (layer 0): HW
  - Highest (layer N): user interface
- Modularity
  - Layers are selected such that each uses functions (operations) and services of only lowerlevel layers
- Pros
  - Simplicity of construction
  - Simplified verification scheme
  - Ease of debugging
  - Can easily update or extend each level
- Cons
  - Difficulty to define layers
  - Layers may need to interact with various layers
  - Too many layers $\rightarrow$ performance degradation

2.16 Microkernel System Structure

- Moves as Much from Kernel into User Space
  - Non-essential components implemented as system and user-level programs
  - Results in much smaller kernel called microkernel
- Microkernel
  - Provide minimal process and memory management and communication facility
- Mach Examples of Microkernel
- Communication Takes Place between User Modules using Message Passing

- Pros
  - Easier to extend a microkernel
  - Easier to port OS to new architectures
  - More reliable (less code running in kernel mode)
  - More secure
- Cons
  - Performance overhead of user space to kernel space communication

2.17 Modules

- Loadable Kernel Modules
  - Components can be loaded either at boot time or during run time
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to others over known interfaces
  - Each is loadable as needed within kernel
- Similar to Layers but with More Flexibility (Linux, Solaris)
- Similar to Microkernels but More Efficient (Modules do not need to invoke message passing to communicate)

2.18 Solaris Modular Approach

- Core Solaris kernel
- Device and bus drivers
- File systems
- Loadable system calls
- Miscellaneous modules
- STREAMS modules
- Executable formats
- Scheduling classes

2.19 Hybrid Systems

- Most Modern OSes actually not One Pure Model
  - Hybrid combines multiple approaches to address performance, security, usability needs
  - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic
  - Windows mostly monolithic, plus microkernel for different subsystem personalities
  - Apple Mac OS X hybrid, Layered
    - Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called kernel extensions)

2.20 Virtual Machines

- Features
  - To abstract HW of a single computer into several different execution environments
  - Creates an illusion that each environment running its own private computer
  - Allows OSes to run applications within other OSes
  - Host OS runs several guest OSes as processes
  - First appeared commercially on IBM mainframes
- Hypervisor (HV): Virtual Machine Monitor
  - SW/HW that creates and runs virtual machines
- Type 1
  - HV runs directly on host's HW to control HW and to manage guest OSes
  - A guest OS thus runs on another level above HV
- Type 2
  - HV runs within a conventional OS environment
  - With HV layer as a distinct second SW level, guest OS run at third level above HW
- Main Benefits
  - Guest OSes do not have to be compliant with HW
  - Host system protected from other VMs
  - Alleviates system development
  - A VM can easily be relocated from one physical machine to another as needed
  - Rapid porting and testing of program
    - Quality assurance engineers can test their programs on multiple platforms on a single HW
  - Can consolidate multiple systems in a single HW
    - Significant power efficiency in data centers
  - Can offer pre-installed applications on VMs
- Virtualization
  - Guest OS & applications run at almost full speed
  - Only system's resources need to be virtualized
  - ISA is not virtualized
- Emulation
  - ISA is virtually implemented (or emulated)
  - Used when source CPU type different from target type
  - Generally slowest method (by 10X)
- Implementation
  - Underlying machine: user and kernel mode
  - HV can run in kernel mode
  - VMs can execute only in user mode
  - VM: virtual user mode and virtual kernel mode both runs in physical user mode
  - This could adversely affect performance
  - This requires some sort of HW mode support for virtualization: called host mode or HV mode

2.21 Docker/Container vs. Virtualization

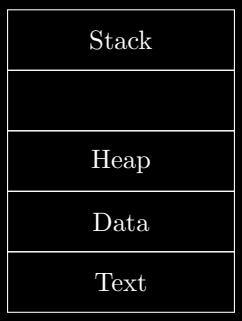
- Container/Docker
  - It does OS virtualization (not HW virtualization)
  - OS-Level virtualization to deliver SW in packages
  - Share one OS (no multiple guest OS)
  - Higher performance
  - Resolves library dependency
  - Less isolation
  - Less security
  - Each container contains only application & its libraries and dependencies
  - Small, fast, and portable
  - A containerized application has more direct access to HW vs. an application running in a VM

3 Processes

3.1 Process Concept

- A Program in Execution
- Process execution must progress in sequential fashion
- A System Consists of a Set of Processes
  - OS processes executing system code
  - User processes executing user code
- Batch system: jobs
- Time-shared systems: user programs or tasks
- Multiple Parts
  - Program code, also called text section
  - Current activity including program counter, processor registers
  - Stack containing temporary data
  - Data section containing global variables
  - Heap containing memory dynamically allocated during run time
- Program is Passive Entity Stored on Disk (executable file), process is active
- Program becomes process when executable file loaded into memory
- Execution of Program Starts via:
  - GUI mouse clicks
  - Command line entry of its name
- One Program can be Several Processes

3.2 Process in Memory



Process in Main Memory

-

3.3 Process State

- As a Process Executes, it Changes State
- New: Process is being created
- Running: Instructions are being executed
- Waiting: Process is waiting for some event to occur
- Ready: Process is waiting to be assigned to a processor
- Terminated: Process has finished execution

3.4 Process Control Block (PCB)

- Info Associate with each Process
  - Aka, task control block
- Process State
  - Running, waiting, ready, new, & terminated
- Program Counter
- CPU Registers
  - Contents of all process-centric registers: General-Purpose registers, Stack register
- CPU Scheduling Info
  - Priorities, scheduling queue pointers
- Memory Management Info
  - Memory allocated to process
  - Page or segment tables
- Accounting Info
  - CPU used
  - Clock time elapsed since start
  - Time limits
- I/O Status Info
  - I/O devices allocated to process
  - List of open files

3.5 Threads

- Originally, Process has a Single Thread of Execution
- Consider Having Multiple Program Counters per Process
  - Multiple locations can execute at once
  - Multiple threads of control $\rightarrow$ threads
- Must then Have Storage for Thread Details
  - Multiple program counters in PCB

3.6 Process Representation in Linux

- Doubly Linked List of Task_Struct
  - Kernel Maintains a Pointer to Current Process

3.7 Process Scheduling

- Process Scheduler
  - Selects among available processes for next execution on CPU
- Possible Goals of Process Scheduling
  - To maximize CPU utilization
  - To meet deadline of each process
  - To meet fairness among processes

- Maintains Scheduling Queues of Processes
  - Job queue: set of all processes in system
  - Ready queue: set of all processes residing in main memory, ready, and waiting to execute
  - Device queues: set of processes waiting for an I/O device (one queue for each device)
- Processes Migrate among Various Queues
  - Job queue → ready queue
  - Device queue → ready queue

3.8 Schedulers

- A Process Migrates Among various Scheduling Queues throughout its Lifetime
- OS selects (schedules) processes in queues
- Short-Term (ST) Scheduler (CPU Scheduler)
  - Selects which process should be executed next and allocates CPU
  - Sometimes the only scheduler in a system
  - Is invoked frequently (must be fast)
- Long-Term (LT) scheduler (job scheduler)
  - Selects which processes should be brought into ready queue
  - Invoked infrequently (may be slow)
  - Controls degree of multiprogramming (determines no of processes in memory)
  - Steady-State: long-term scheduler should be invoked only a process leaves the system (Assuming degree of multiprogramming is stable)
- Processes can be Described as Either:
  - I/O-bound process: spends more time doing I/O than computations
  - CPU-bound process: spends more time doing computations
- LT Scheduler Strives for Good process mix
  - All processes I/O bound → ready queue will almost always be empty → ST scheduler idle
  - All processes CPU bound → I/O waiting queue will almost always be empty → devices idle
- LT Scheduler May be Absent or Minimal
  - Can adversely affect performance
  - So, users may quit and their processes are terminated

3.9 Medium Term Scheduling

- Medium-Term Scheduler Used if Degree of Multi-Programming Needs to Decrease
- Swapping: remove process from memory, store on disk (swapped out), bring back in from disk to continue execution (swapped in)
- Used to improve process mix

3.10 Context Switch

- When CPU Switches to another Process
  - System must save state of old process
  - OS, then, loads saved state for new process via a context switch
- Context of a Process Represented in PCB
- Context Switch Time is Overhead
  - System does no useful work while switching
  - More complex OS and PCB → longer context switch
  - More context switching time when using VMs
- Context Switch Time Depends on HW Support
  - Memory speed
  - Number of registers in RF
  - Special instruction to copy RF
  - Some HW provides multiple sets of registers per CPU → multiple contexts loaded at once
    - Context switch: changing a pointer to a target RF

3.11 Operations on Processes

- System must Provide Mechanisms for
  - Process creation
  - Process termination

3.12 Process Creation

- Parent Process Creates Children processes
  - Which, in turn creates other processes, forming a tree of processes
- Process Identified and Managed via a Process Identifier (pid)
- Root Process in Linux: "init"
  - Daemon processes: "login", "kthreadd", "sshd",...
- Root Process in Solaris: "sched"
  - Children processes: "Init" ("Init" root of user processes), "pageout", "fsflush",...
- Resource Sharing Options
  1. Parent and children share all resources
  2. Children share subset of parent's resources
     - Prevents any process from overloading system by creating too many sub-processes
  3. Parent and child share no resources
- Execution Options
  1. Parent and children execute concurrently
  2. Parent waits until children terminate
- Address Space
  1. Child duplicate of parent
  2. Child has a new program loaded into it
- `fork()`

- System call creates new process
- New process consists of a copy of address space of original process
- Convenient communication between two processes
- What happens after a `fork()`?
  - Both processes continue execution at the instruction after `fork()`
  - Child gets unique process ID
  - Child's PPID = parent's PID
  - Reset child's resource utilization and CPU time counters
- Return value
  - `fork()` > 0
    - Running in (original) Parent process
    - return value is pid of new child
  - `fork()` $= 0$
    - Running in new Child process
  - `fork()` < 0
    - Error! Must handle somehow
    - Running in original process

- `exec()`
  - system call used after a `fork()` to replace process' memory space with a new program
- What Does not Child Inherits from Parent?
  - Parents memory locks
  - Parent's timers
  - Semaphore's adjustments and pending signals
- What Child Process Inherits from Parent?
  - Privileges and scheduling attributes
  - Certain resources such as open files
  - Address space

3.13 Process Termination

- Process Executes Last Statement
  - Then asks OS to delete it
  - Using `exit()`
  - Returns status data from child to parent via `wait()`
  - Process' resources are de-allocated by OS
- Parent may Terminate Execution of Children Processes Using `abort()`
- Reasons to Terminate Child Processes
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - Parent is exiting and OS does not allow a child to continue if its parent terminates
- Some OSes do not Allow Child to Exists, if its parent has terminated
  - If a process terminates, then all its children must also be terminated
  - Cascading termination
  - All children, grandchildren, etc. are terminated
  - Termination is initiated by OS

- Parent process may Wait for Termination of a Child Process by Using `wait()`
  - Call Returns Status Info and pid of Terminated Process
  - `pid=wait(&status)`
- Zombie Process
  - If no parent waiting (did not invoke `wait()`)
  - Process has completed execution but still has an entry in process table
  - Entry needed for possible reading of exit status by its parent
- Orphan Process
  - Parent terminated without Invoking `wait()`
  - In UNIX, any orphan process is immediately adopted by "init" process
    - Called "reparenting"
  - A process can become orphan intentionally or unintentionally
    - Process crash
    - To run a process indefinitely (in the background)
- Other Variations of Waiting
  - `waitid()`
  - `waitpid()`

3.14 Multiprocess Architecture – Chrome Browser

- Many Web Browsers Ran as Single Process (Some Still Do)
- If one web-site causes trouble, entire browser can hang or crash
- Google Chrome Browser is Multi-Process with Three Different Types of Processes
  - Browser process manages user interface, disk and network I/O
  - Renderer process renders web pages, deals with HTML, Javascript
    - A new renderer created for each website opened
    - Runs in sandbox restricting disk and network I/O, minimizing effect of security exploits
  - Plug-in process for each type of plug-in

3.15 Interprocess Communication

- Independent Process
  - Cannot affect or be affected by execution of another Process
  - Does not share data with any other processes
- Cooperating Process can Affect or be Affected by Execution of another Process
  - Motivations of Process Cooperation
    - Information sharing
    - Computation speed-up
      - Breaking into subtasks & executing on multiple cores
    - Modularity
      - Constructing a system in a modular fashion
      - Dividing system functions into separate processes
    - Convenience
      - Several tasks of a single user (editing, printing, . . .)
- Cooperating Processes need Interprocess Communication (IPC)
- Two Models of IPC:

    - Shared memory
    - Message passing
- Most OSes Implement Both Models
- Shared memory
    - Maximum speed
    - Convenience of communication
    - System calls used only to establish shared-memory regions → Thereafter, all accesses are treated as normal memory accesses
- Message passing
    - Useful for exchanging smaller amounts of data
    - No conflicts need to be avoided
    - Easier to be implemented for inter-computer communication
    - Implemented by system calls → time-consuming

3.16 Shared-Memory Solution: Producer-Consumer Problem

- Paradigm for Cooperating Processes
    - Producer process produces info that is consumed by a consumer process
    - Unbounded-buffer places no practical limit on size of buffer
    - Bounded-buffer assumes that there is a fixed buffer size

3.17 Shared Memory

- An area of memory shared among processes that wish to communicate
- Communication is under Control of Users Processes not OS
- Major Issue
    - To provide mechanism that will allow user processes to synchronize their actions when they access shared memory

3.18 Message Passing

- Mechanism for Processes to Communicate and to Synchronize their Actions
- Processes Communicate with each other
    - Without resorting to shared variables
    - Without having a shared address space
- Significantly useful in a Distributed Systems
- Two Major Operations
    - `send(message)`
    - `receive(message)`
- If Processes P and Q Wish to Communicate, they need to:
    - Establish a communication link between them
    - Exchange messages via send/receive
- Message size is either Fixed or Variable
- Fixed Message Size
    - Straightforward implementation
    - Difficulty for programmer

3.19 Communication Link

- Communication Link can be Viewed at either Physical or Logical
- Physical
  - Shared memory
  - Hardware bus
  - Network
- Logical
  - Direct or indirect
  - Synchronous or asynchronous
  - Automatic or explicit buffering

3.20 Direct Communication

- Processes must Name each other Explicitly
  - `send (P, msg)`: send a message to process P
  - `receive(Q, msg)`: receive a message from Q
- Properties of Direct Communication Link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - Link may be unidirectional, but is usually bidir

3.21 Indirect Communication

- Main Issue in Direct Communication
  - Identifiers must explicitly stated
- Messages are Directed and Received from Mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of an Indirect Communication Link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bidirectional
- Operations
  - Create a new mailbox (port)
  - Send and receive messages through mailbox
  - Destroy a mailbox
- Primitives are Defined as:
  - `send(A, message)`: send a message to mailbox A
  - `receive(A, message)`: receive a message from mailbox A
- Mailbox Sharing
  - P1, P2, and P3 share mailbox A
  - P1, sends; P2 and P3 receive

- Who gets message?
- Allow a link to be associated with at most two processes
- Allow only one process at a time to execute a receive operation
- Allow system to select arbitrarily receiver
- Sender is notified who receiver was

3.22 Synchronization

- Message Passing may be either Blocking or Non-Blocking
- Blocking considered Synchronous
  - Blocking send: sender is blocked until message is received
  - Blocking receive: receiver is blocked until a message is available
- Non-Blocking Considered Asynchronous
  - Non-blocking send: sender sends message and continue
  - Non-blocking receive: receiver receives
- Different Combinations Possible
  - If both send and receive are blocking $\rightarrow$ we have a rendezvous

3.23 Buffering

- mplemented in one of Three Ways
  1. Zero capacity: no messages are queued on a link
     - Sender must wait for receiver (rendezvous)
  2. Bounded capacity: finite length of n messages
     - Sender must wait if link full
  3. Unbounded capacity: infinite length
     - Sender never waits

3.24 Linux Pipes

- Linux `pipe()` system call
- Allows anonymous (un-named) communication
- Unidirectional
- Produces/consumes data through file (`write()` and `read()`)
- Data stored in kernel via pipefs in virtual filesystem

4 Threads

4.1 Multithreaded Server Architecture

1. Client sends request to server
2. Server creates new thread to handle the request
3. Server resumes listening for new client requests

4.2 Single and Multithreaded Processes

Code	Data	Files
Registers		Stack

Code	Data	Files
Registers		Stack

Multiple Processes with Single Thread

-
- Code, Data, Files, Registers, and Stack are the same with in a process.

Code	Data	Files
Registers	Registers	Registers
Stack	Stack	Stack

Code	Data	Files
Registers	Registers	Registers
Stack	Stack	Stack

Multiple Processes with Multi thread

-
- Code, Data, and Files are the same with in a process. Register and Stack is the same with in a thread.

4.3 Multi-Threading in OS Kernels

- Most OSes are Multi-Threaded
- Each thread performs a specific task
    - A thread to manage I/O type A
    - A thread to manage memory
    - A thread for interrupt handling

4.4 Benefits of Multi-Threading

- Responsiveness
  - May allow continued execution if part of process is blocked, especially important for user interfaces
- Resource Sharing
  - Threads share resources of process, easier than shared memory or message passing
- Economy
  - Cheaper than process creation, thread switching lower overhead than context switching
  - Solaris: process creation 30X slower, context switching 5X slower
- Scalability
  - Process can take advantage of multiprocessor architectures

4.5 Threads vs. Processes

- Processes typically Independent
  - Threads exist as subsets of a process
- Processes Carry Much more State Info
  - Threads of a process share process state as well as memory and other resources
- Processes have Separate Address Spaces
  - Threads share their address space
- Processes Interact Only through IPC
- Processes Much Slower Context Switch Time
- Processes Much Slower Creation Time

4.6 Multicore Programming

- Multicore or Multiprocessor Systems Putting Pressure on Programmers
- Programming Challenges Include:
  - Dividing activities
  - Balance
  - Data splitting
  - Data dependency
  - Testing and debugging
- Parallelism
  - Implies a system can perform more than one task simultaneously
- Concurrency
  - Supports more than one task making progress
  - Single processor (core), scheduler providing concurrency
  - Possible to have concurrency without parallelism
- Before Advent of Multi-core Systems
  - CPU schedulers designed to provide illusion of parallelisms (not actual parallelism)
  - Processes were running concurrently (but not in parallel)
- As number of Threads Grows, so Does Architectural Support for Threading
  - CPUs have cores as well as HW threads
    - HW support such as multiple sets of RFs and PCs
  - Consider Oracle SPARC T4 with 8 cores, and 8 hardware threads per core

4.7 Concurrency vs. Parallelism

- Concurrent Execution on a Single-Core System:
  - $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1 \rightarrow \ldots$
- Parallelism on a Multi-Core System:
  - $T_1 \rightarrow T_3 \rightarrow T_1 \rightarrow T_3 \rightarrow T_1 \rightarrow \ldots$
  - $T_2 \rightarrow T_4 \rightarrow T_2 \rightarrow T_4 \rightarrow T_2 \rightarrow \ldots$

4.8 Amdahl's Law

- Imagine an Application has both Serial and Parallel Components

$$speedup \leq \frac{1}{S + \frac{1-S}{N}} \tag{1}$$

- S is Serial Portion
- N Processing Cores

4.9 User Threads and Kernel Threads

- Kernel Threads
  - Supported and managed directly by OS
- User Threads
  - Management done by user-level threads library
  - Supported above kernel and managed without kernel support
    - Kernel is unaware of them unless they are mapped to kernel threads
  - Context switching very fast (no interaction with kernel)
- Ultimately, a Relationship Must Exist between User threads and Kernel threads

4.10 Multithreading Models

- Many-to-One (N:1)
  - All application-level threads maps to a single kernel-level scheduled entity
- One-to-One (1:1)
  - Threads created by user are in 1-1 correspondence with schedulable entities in kernel (Some say, no user-level thread)
- Many-to-Many (M:N)
  - Maps M number of application-level threads to N number of kernel entities

4.11 Many-to-One

- Many User-Level Threads Mapped to Single Kernel Thread
- Thread management Done by thread library in user space $\rightarrow$ fast & efficient
- Entire Process Will Block if a thread Makes a Blocking System Call
  - Alternatively, a single thread can monopolize time slice
- Multiple Threads may not Run in Parallel on Multicore
  - Since only one may be in Kernel at a Time
  - Does not benefit from multicore
- Few Systems Currently use this Model

4.12 One-to-One

- Each User-Level Thread Maps to Kernel Thread
- Creating a User-Level Thread Creates a Kernel Thread
- More Concurrency than Many-to-One
  - Allows another thread to run when a thread makes a blocking system call
- Main Drawback
  - Creating a user thread requires creating corresponding kernel thread
  - Overhead of creating kernel threads can burden performance of an application
  - For the solution, most implementations of this model restrict number of threads supported by system

4.13 Many-to-Many

- Multiplexes Many User Level Threads to be Mapped to Smaller or Equal Number of Kernel Threads
- Allows OS to Create a Sufficient Number of Kernel Threads
- Tries to Address Shortcomings of Many-to-one and One-to-One Models

4.14 Two-level Model

- Similar to M:M, Except that it Allows a User Thread to be Bound to Kernel Thread

4.15 Processes, User Threads, Kernel Threads

- Process
  - Isolated with its own virtual address space
  - Contains process data like file handles
  - Lots of overhead
  - Every process has at least one kernel thread
- Kernel Threads
  - Shared virtual address space
  - Contains running state data
  - Less overhead
  - From OS's point of view, this is what is scheduled to run on a CPU
- User Threads
  - Shared virtual address space, contains running state data
  - Kernel unaware
  - Even less overhead

4.16 Trade-Offs

- Processes
  - Secure and isolated
  - Kernel aware
  - Creating a new process (address space!) brings lots of overhead
- Kernel Threads
  - No need to create a new address space
  - No need to change address space in context switch
  - Kernel aware

    - Still need to enter kernel to context switch
- User Threads
    - No new address space, no need to change address space
    - No need to enter kernel to switch
    - Kernel is unaware, No multiprocessing, I/O blocks all user threads

4.17 Thread Libraries

- Thread library Provides Programmer with API for Creating and Managing Threads
- Two Primary Ways of Implementing
    - Library entirely in user space
    - Kernel-level library supported by OS
- Library Entirely in User Space:
    - With no kernel support
    - All code/data structures in user space
    - Invoking a function in API results in a local function call in user space not a system call
- Kernel-Level Library Supported by OS
    - Code and data structures for library exist in kernel space
    - Invoking a function in API $\rightarrow$ system call
- Three Main Thread Libraries in Use
    - POSIX Pthreads Library
    - Win32 Thread Library
        - Kernel-level library
    - Java Thread API
        - Allows threads to be created & managed directly in Java programs
        - Implemented using a thread library of host system since JVM runs on top of host system

4.18 Pthreads

- May be Provided either as User-Level or Kernel-Level
- A POSIX Standard API for Thread Creation and Synchronization
- Specification, not Implementation
- API Specifies Behavior of Thread Library, Implementation is up to Development of Library
- Common in UNIX OS (Solaris, Linux, Mac OS X)

4.19 Java Threads

- Java Threads Managed by JVM
- Typically Implemented using Threads Model provided by Underlying OS
- Each Java Program consists of at Least One Single Thread

4.20 Implicit Threading

- Growing in popularity as the number of threads increases; program correctness becomes more difficult with explicit threads.
- Creation and management of threads is done by compilers and run-time libraries rather than programmers.

4.21 Threading Issues

- Semantics of `fork()` and `exec()` system calls
  - Does `fork()` Duplicate only Calling Thread or all Threads?
    - Some UNIXes have two versions of fork
  - What about exec?
    - Usually works as normal
    - Replaces running process including all threads
- Signal Handling
  - Synchronous and asynchronous
  - Signals Used in UNIX to Notify a Process that a Particular Event has Occurred
  - SW generated interrupts sent to a process when an event happens
  - Signal Handler Used to Deal with Signals
    - Signal is generated by particular event
    - Signal is delivered to a process
    - Signal is handled by one of two signal handlers:
      - default
      - user-defined
  - Every Signal has Default Handler that Kernel Runs when Handling Signal
    - User-defined signal handler can override default
    - For single-threaded, signal delivered to process
  - Default Actions:
    - Signal discarded after being received
    - Process terminated after signal received
    - A core file written, then process terminated
    - Stop process after signal received
  - Where should a Signal be Delivered for Multi-Threaded?
    - Deliver signal to thread to which signal applies
    - Deliver signal to every thread in process
    - Deliver signal to certain threads in process
    - Assign a specific thread to receive all signals for process
- Thread Cancellation of Target Thread
  - Terminating a Thread before it has finished
  - Thread to be Canceled is Target Thread
  - Two General Approaches:
    - Asynchronous cancellation terminates target thread immediately
    - Deferred cancellation allows target thread to periodically check if it should be cancelled
  - Invoking Thread Cancellation Requests Cancellation, but Actual Cancellation Depends on Thread State
  - If Thread has Cancellation Disabled, Cancellation Remains Pending until Thread Enables it
  - Default Type is Deferred
    - Cancellation only occurs when thread reaches cancellation point
  - On Linux, thread cancellation handled through signals
- Scheduler Activations

4.22 Thread Pools

- Create a Number of Threads in a Pool where they Await Work
- Advantages:
  - Usually slightly faster to service a request with an existing thread than create a new thread
  - Allows number of threads in application(s) to be bound to size of pool
- Windows API supports thread pools

4.23 Scheduler Activations

- Many-to-Many and Two-Level Models:
  - Require communication to maintain appropriate number of kernel threads allocated to application
- LightWeight Process (LWP)
  - Typically use an intermediate data structure between user and kernel threads
  - Appears to be a virtual processor on which process can schedule user thread to run
  - Each LWP attached to kernel thread

5 CPU Scheduling

5.1 Basic Concepts

- Maximum CPU Utilization Obtained with Multiprogramming
- CPU–I/O Burst Cycle
  - Process execution consists of a cycle of CPU execution and I/O wait
- CPU Burst followed by I/O Burst
- Major Concerns
  - CPU burst duration
  - CPU burst distribution

5.2 CPU Scheduler

- Short-Term Scheduler selects from among processes in ready queue, and allocates CPU to one of them
  - Queue may be ordered in various ways
- CPU Scheduling Decisions may Take Place when a process:
  - Switches from running to waiting state
  - Switches from running to ready state
  - Switches from waiting to ready state
  - Terminates
- Non-Preemptive Scheduling
  - AKA, cooperating scheduling
  - Simple to implement
  - Low context switch flexibility
  - Process hogging very likely to happen
- All other Scheduling is Preemptive
  - More flexibility in context switching
  - More complex to implement
  - More issues for implementation
    - Consider access to shared data
    - Consider preemption while in kernel mode
    - Consider interrupts occurring during crucial OS activities

5.3 Dispatcher

- Dispatcher Module
  - Gives Control of CPU to process Selected by short-term scheduler
- Dispatcher Involves:
  - Switching context
  - Switching to user mode
  - Jumping to proper location in user program to restart that program
- Dispatch latency
  - Time it takes for dispatcher to stop one process and start another running

5.4 Scheduling Criteria

- CPU Utilization: Keep CPU as busy as possible
- Throughput: Number of processes that complete their execution per time unit
- Turnaround Time: Amount of time to execute a particular process
- Waiting Time: Amount of time a process has been waiting in ready queue
- Response Time: Amount of time it takes from when a request was submitted until first response is produced, not output (for time-sharing environment)

5.5 Scheduling Algorithm Optimization Criteria

- Max CPU Utilization
- Max Throughput
- Minimize Response Time
  - Average response time
  - Maximum response time
  - Variance of response time
- Minimize Turnaround Time
  - Average, maximum, variance
- Minimize Waiting Time
  - Average, maximum, variance

5.6 First Come, First Served (FCFS) Scheduling

- Non-preemptive $\rightarrow$ not suited for timesharing or real-time systems
- Convoy Effect: short process behind long process
  - Consider one CPU-bound and many I/O-bound processes

5.7 Shortest Job First (SJF) Scheduling

- Associate with each Process Length of its Next CPU Burst
- Use these lengths to schedule process with shortest time
- Gives minimum average waiting time for a given set of processes
- Difficulty is knowing length of next CPU request
- Possible starvation

5.8 Determining Length of Next CPU Burst

- Can only Estimate Length, should be Similar to Previous one
- Then pick process with shortest predicted next CPU burst
- Can be done by using length of previous CPU bursts, using exponential averaging

$$\tau_{n+1} = \alpha\tau_n + (1-\alpha)\tau_n \tag{2}$$

5.9 Shortest Remaining Time First

- Preemptive version of SJF

5.10 Priority Scheduling

- A Priority Number (Integer) is Associated with each Process
- CPU is allocated to Process with Highest Priority (smallest integer is highest priority)
- SJF is Priority Scheduling where Priority is Inverse of Predicted Next CPU Burst Time
- Starvation: low priority processes may never execute
- Aging: as time progresses increase process priority

5.11 Round Robin (RR)

- Each Process Gets a Small Unit of CPU Time (time quantum q)
  - Usually 10-100 milliseconds
  - After this time has elapsed, process is preempted and added to end of ready queue
  - No process waits more than $(n-1)q$ time units
- Timer Interrupts every Quantum to Schedule next Process
- q very large is FCFS
- q must be large with respect to context switch, otherwise overhead is too high
- Higher average turnaround than SJF
- Better response
- Rule of Thumb: 80% of CPU bursts should be shorter than q

5.12 Multilevel Queue

- Processes can be Classified into Different Groups
- Processes have different response-time requirements $\rightarrow$ different scheduling needs
- ready queue partitioned into two queues Foreground (interactive), Background (batch)
- Processes Permanently Assigned to one Queue
- Each Queue has its own Scheduling Algorithm
  - Foreground (RR)
  - Background (FCFS)
- Scheduling must be done between Queues
  - Fixed priority scheduling
  - Time slice: each queue gets a certain amount of CPU time which it can schedule amongst its processes

5.13 Multilevel Feedback Queue

- A Process can Move between Various Queues
  - Aging can be implemented this way
- Multilevel-Feedback-Queue Scheduler defined by following Parameters
  - Number of queues
  - Scheduling algorithms for each queue
  - Priority among queues (fixed, time slice, etc.)
  - Method used to determine when to upgrade a process
  - Method used to determine when to demote a process
  - Method used to determine which queue a process will enter when that process needs service

5.14 Thread Scheduling

- Distinction between User- and Kernel-Level Threads
- When Threads Supported, Threads Scheduled, Not Processes
- Many-to-One and Many-to-Many Models
  - Thread library schedules user-level threads to run on LWP (still may not be running on CPU)
  - Known as process-contention scope (PCS) since scheduling competition is within process
  - Typically done via priority set by programmer
- Kernel Thread Scheduled onto Available CPU
  - Aka, System-Contention Scope (SCS)
  - Competition among all threads in system

5.15 Pthread Scheduling

- API allows Specifying either PCS or SCS during Thread Creation
- `PTHREAD_SCOPE_PROCESS` schedules threads using PCS scheduling
  - Schedules user-level threads onto available LWPs on systems implementing many-to-many model
- `PTHREAD_SCOPE_SYSTEM` schedules threads using SCS scheduling
  - Binds an LWP for each user-level thread on many-to-many systems (similar to one-to-one policy)
- Can be Limited by OS: Linux and Mac OS X only allow `PTHREAD_SCOPE_SYSTEM`
  - When one-to-one model is used

5.16 Multiple-Processor Scheduling

- CPU Scheduling more Complex when Multiple CPUs are Available
- Homogeneous Processors within a Multiprocessor
  - As apposed to heterogeneous architecture
- Asymmetric Multiprocessing
  - All scheduling decisions, I/O processing, and system activities handled by a single processor
    - Hence, only one processor accesses system data structures, alleviating need for data sharing
    - Simple design & implementation
- Symmetric Multiprocessing (SMP)
  - Each processor is self-scheduling
  - All processes in common ready queue OR each has its own private queue of ready processes
  - Currently, most common
  - Supported by all well-known OSs
    - WinXP, Win 2000, Solaris, Linux, Mac OS X
- Processor Affinity: Process has affinity for processor on which it is currently running
- Why to Have Affinity?
  - Process migration very costly; it requires:
    - Cache invalidation in source processor
    - Cache repopulation in destination processor
- Types of Affinity
  - Soft affinity
  - Hard affinity
    - Processor sets

5.17 NUMA and CPU Scheduling

- Main Memory can affect processor affinity
- Memory placement algorithms can consider affinity

5.18 Multiple-Processor Scheduling – Load Balancing

- If SMP, Need to Keep all CPUs Loaded for Efficiency
- Load Balancing attempts to keep workload evenly distributed; two approaches as follows:
  - Push Migration: periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
  - Pull Migration: idle processors pulls waiting task from busy processor
  - Load balancing may contradict processor affinity

5.19 Multicore Processors

- Recent Trend to Place Multiple Cores on Same Physical Chip
  - Faster and consumes less power
- Memory Stall Significantly Affects Performance
- Multiple Threads per Core also Growing
  - E.g., UltraSPARC T1: 8 cores, 4 threads per core $\rightarrow$ 32 logical processors from OS perspective
  - Takes advantage of memory stall to make progress on another thread while memory retrieve happens
- Memory stall cannot be handled by OS $\rightarrow$ Needs HW threading support to switch to another thread to keep CPU cores as busy as possible

5.20 Conclusion: Levels of Scheduling

- Three Levels of Scheduling
- L2: Scheduling User Threads to Kernel Threads (Scheduling user threads to LWPs)
  - Managed by threading lib or application programmer
  - Not exist in one-to-one models
- L1: Scheduling Kernel Threads to HW Threads
  - Aka, logical Processors
  - Typically managed by OS
- L0: Scheduling HW Threads for Execution on HW Cores
  - Managed by HW controller
- User Thread $\rightarrow$ Kernel Thread $\rightarrow$ HW Thread $\rightarrow$ Core $\rightarrow$ Processor

5.21 Real-Time CPU Scheduling

- Real-Time System
  - Results must be produced within a specified deadline period
  - In addition, computing results must be correct
  - Can present obvious challenges
- Soft Real-Time Systems
  - Only guarantees that a process will be given a preference over non-critical processes
  - No guarantee as to when critical real-time process will be scheduled

- Hard Real-Time Systems
  - Task must be serviced by its deadline
- Two Types of Latencies affect Performance
  - Interrupt latency and dispatch latency
- Interrupt latency
  - Time from Arrival of interrupt to start of routine that services interrupt
  - Completing current instruction
  - Determine type of interrupt
  - Save state of current process
  - Interrupt disabled time interval
    - Amount of time interrupts may be disabled While kernel data structures are being updated
- Dispatch latency
  - time for schedule to take current process off CPU and switch to another
  - Effective approach is using preemptive kernel
- Issues in Dispatch Latency
  - Preemption of any process running in kernel mode
  - Release by low-priority process of resources needed by high-priority processes
- Effect of Preemption Disabled
  - In solaris, dispatch latency over 100 ms
  - When enabled, less than 1 ms

5.22 Features of Real-Time Kernels

- Windows XP over 40 Million lines of Source Code
- A RT OS very Simple Design
  - Written in 10K 100K LoC
  - Not supporting many features of typical OSes
  - Variety of peripheral devices, protection/security mechanisms, and multiple users
- Why RT OS does not Provide Advanced Features?
  - RT OSes are designed for single-purpose apps
  - Additional features require large memory and fast CPU
  - Supporting advanced features increases cost of RT systems

5.23 Implementing RT OS

- RT OS Must Support & Implement Three Features
  - Preemptive, priority-based scheduling
    - Can assign different priorities to processes
    - A process currently running on CPU will be preempted if a higher-priority process arrives
    - Most OSes support priority scheduling
      - Win-XP has 32 different priority levels
    - This feature only guarantees soft RT functionality
  - Preemptive kernel
    - Non-preemptive kernels disallow preemption of a process running in kernel mode
    - A kernel-mode process will run until it exists kernel mode

        - Designing preemptive kernels is quite difficult
        - Desktop OSes such as Win-XP are non-preemptive since apps such as word or excel do not require such quick response
        - Preemptive kernels is mandatory in hard RT systems (e.g., RTLinux)
    - Minimized latency
        - Try to minimize response time of events
        - Different events require different latency requirements
- Well-Known RT OS:
    - VxWorks (1987, last release: Feb. 2024)
    - QNX (1982, last release: Dec. 2023)
    - Free RTOS (2003, last release: April 2024)
    - LynxOS (1986, last release: Feb. 2020)
    - Windows CE (1996, last release: 2013)

5.24 Priority-based Scheduling

- For Real-Time Scheduling, Scheduler must support Preemptive, Priority-based Scheduling
    - But only guarantees soft real-time
- For Hard Real-Time must also Provide ability to Meet Deadlines
- Processes have new Characteristics: periodic ones require CPU at Constant intervals
    - Has processing time t, deadline d, period p
    - $0 \leq t \leq d \leq p$
    - Rate of periodic task is $\frac{1}{p}$
- Admission Control: either admits or rejects request

5.25 Rate Monotonic Scheduling

- A Priority is Assigned based on Inverse of its Period (Static Priority policy)
- Shorter periods $\rightarrow$ higher Priority
- Longer periods $\rightarrow$ lower Priority

5.26 Earliest Deadline First Scheduling (EDF)

- Priorities are Assigned according to Deadlines (Dynamic Scheduling Policy)
    - Earlier deadline, higher priority
    - Later deadline, lower priority

5.27 POSIX Real-Time Scheduling

- POSIX.1b standard
- API provides functions for managing real-time threads
- Defines two scheduling classes for real-time threads:
    - `SCHED_FIFO`: threads are scheduled using a FCFS strategy with a FIFO queue. There is no time-slicing for threads of equal priority
    - `SCHED_RR`: similar to `SCHED_FIFO` except time-slicing occurs for threads of equal priority
- Defines Two Functions for Getting and Setting Scheduling policy
    - `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`
    - `pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)`

6 Memory

6.1 Base and Limit Registers

- A Pair of Base and Limit Registers Define Logical Address Space
- CPU must Check Every Memory Access
  - Generated in user mode to be sure it is between base and limit for that user
  - Trap to operating system monitor—addressing error

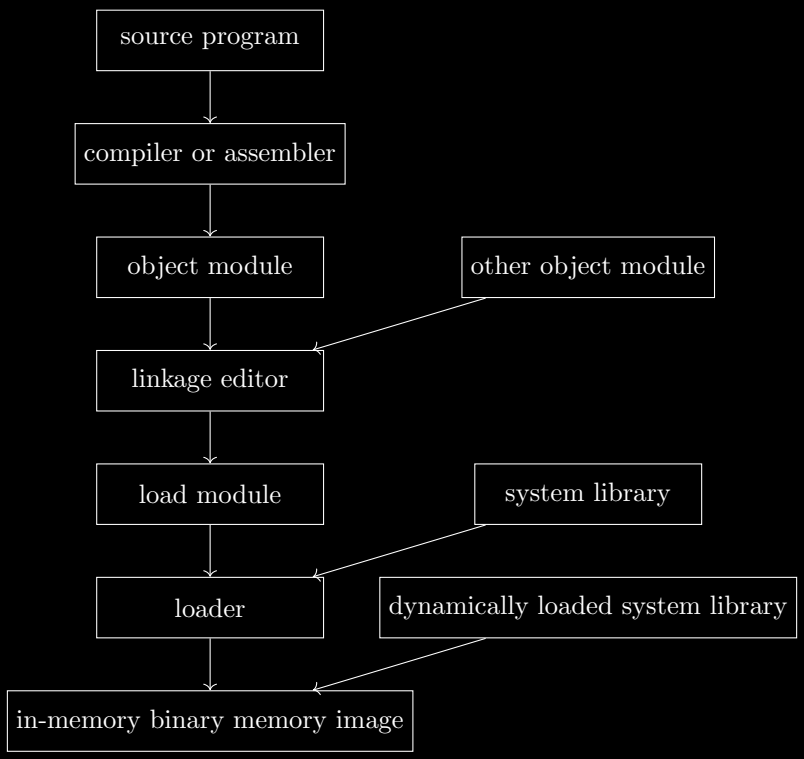
6.2 Address Binding

- Programs on Disk, Ready to be Brought into Memory to Execute Form an Input Queue
- Without support, must be loaded into address 0000
- Inconvenient to Have First User Process Physical Address Always at 0000
- Further, Addresses Represented in Different Ways at Different Stages of a Program's Life
  - Source code, compiled code, linker/loader addresses
- Different Ways of Addresses Representation
  - Source code addresses usually symbolic
  - Compiled code addresses bind to relocatable addresses
  - Linker or loader will bind relocatable addresses to absolute addresses
  - Each binding maps one address space to another

6.3 Binding of Instructions and Data to Memory

- Address Binding to Memory Addresses can Happen at Three Different Stages
  - Compile time: If memory location known a priori, absolute code can be generated
    - Must recompile code if starting location changes
  - Load time: Must generate relocatable code if memory location is not known at compile time
  - Execution time: Binding delayed until run time if process can be moved during its execution from one memory segment to another
  - Need HW support for address maps (e.g., base and limit registers)

6.4 Multistep Processing of a User Program



Multistep Processing of a User Program

-

6.5 Logical vs. Physical Address Space

- Concept of a Logical Address Space
  - Central to Proper Memory Management
  - Bound to a Separate Physical Address Space
- Logical address
  - Generated by CPU
  - Aka, virtual address
- Physical Address
  - Address seen by memory unit
- Logical and Physical Addresses
  - Same in compile-time and load-time address-binding schemes
  - Differ in execution-time address-binding scheme
- Logical Address Space
  - Set of all logical addresses generated by a program
- Physical Address Space
  - Set of all physical addresses generated by a program
- Memory Management Unit (MMU)
  - HW device that at run-time maps virtual to physical address
- Consider Simple Scheme

    - Value in base register is added to every address generated by a user process at time it is sent to memory
    - Base register now called relocation register
    - MS-DOS on Intel 80x86 used for relocation registers
- User Program Deals with Logical Addresses
    - It never sees real physical addresses
    - Execution-time binding occurs when reference is made to location in memory
    - Logical address bound to physical addresses



Multistep Processing of a User Program

    -

6.6 Dynamic Relocation using a Relocation Register

- Routine not Loaded until it is Called
- Better Memory-Space Utilization
    - Unused routine never loaded
- All Routines Kept on Disk in Relocatable Load Format
- Useful when Large Amounts of Code are Needed to Handle Infrequently Occurring Cases
- No Special Support from OS required
    - Implemented through program design
    - OS can help by providing libraries to implement dynamic loading

6.7 Dynamic Linking

- Static Linking
    - system libraries and program code combined by loader into binary program image
    - Wastes both disk space and main memory
- Dynamic Linking
    - linking postponed until execution time
    - Small piece of code, Stub, used to locate appropriate memory-resident library routine
    - Stub replaces itself with address of routine, and executes routine
- OS Checks if Routine is in Memory
    - If not in address space, add to address space
- Dynamic Linking is Particularly Useful for Libraries
    - Processes that use a language library execute only one copy of library code
    - Libraries updates will be automatically applied

6.8 Swapping

- A Process can be Swapped Temporarily out of Memory to a Backing Store, and Brought back into Memory for Continued Execution
- Why Swap out?
  - Quantum of round robin expired → need to bring a new process → not enough memory space
  - Total physical memory space of processes can exceed physical memory
- Backing store
  - Fast disk large enough to accommodate copies of all memory images for all users
- Major part of Swap Time is Transfer Time
  - Total transfer time is directly proportional to amount of memory swapped
- System Maintains a Ready Queue of ready-to-run processes
  - Which have memory images on disk (or in main memory)

6.9 Context Switch Time and Swapping

- Next Processes to be Put on CPU Not in MM
  - Need to swap out a process & swap in target process
- Context Switch Time can then be Very High
- Does Swapped out Process Need to Swap Back into Same Physical Addresses?
  - Depends on address binding method
  - Standard Swapping is Too Time-Consuming
  - Modified Versions of Swapping Found on Many Systems (i.e., UNIX, Linux, and Windows)
    - Swapping normally disabled
    - Started if more than threshold amount of memory allocated
    - Disabled again once memory demand reduced below threshold
- Reduction of Context Switch Time
  - Reduction size of memory swapped
  - By knowing how much memory really being used
  - System calls to inform OS of memory use via `request_memory()` and `release_memory()`
- Other Constraints as well on Swapping
  - Can't swap out Pending I/O as I/O would occur to wrong process
  - Always transfer I/O to kernel space, then to I/O device known as double buffering, adds overhead

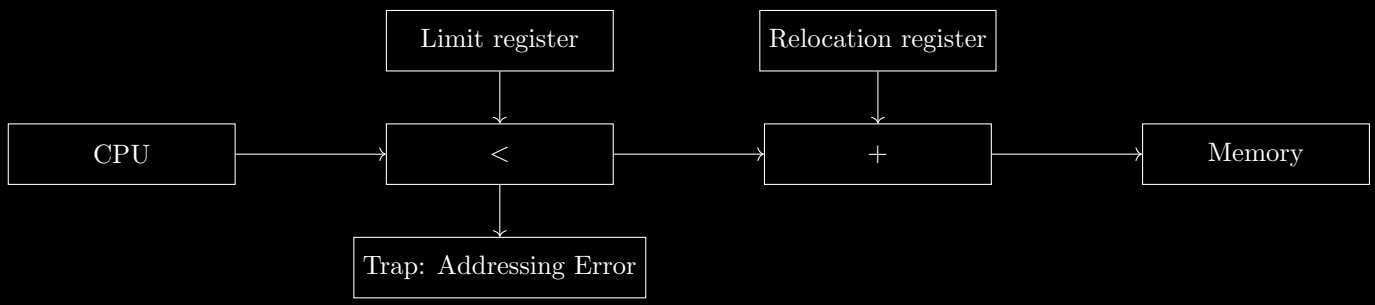
6.10 Memory Management Schemes

- Main Memory must support OS + User Processes
  - Limited Resource èmust allocate efficiently
- Memory Management Schemes
  - Contiguous Allocation
  - Segmentation
  - Paging
  - Segmentation and Paging

6.11 Contiguous Allocation

- Contiguous Allocation is one Early Method
- Main Memory Usually into two partitions:
  - Resident OS, usually held in low memory with interrupt vector
  - User processes then held in high memory
  - Each process contained in single contiguous section of memory
- Relocation Registers used to Protect User Processes from each other, and from Changing OS code and data
  - Base register contains value of smallest physical address
  - Limit register contains range of logical addresses and each logical address must be less than the limit register
  - MMU maps logical address dynamically
  - Can then allow actions such as kernel code being transient and kernel changing size

6.12 HW Support for Relocation and Limit Registers



HW Support for Relocation and Limit Registers

-

6.13 Multiple-Partition Allocation

- Fixed-Sized Partitions
  - Each partition only one process
  - Degree of multiprogramming limited by number of partitions
  - Originally used in IBM OS/360
  - No longer in use
- Variable-sized Partition Scheme
  - Sized to a given process' needs
  - Hole: block of available memory
    - Holes of various size are scattered throughout memory
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Process exiting frees its partition
    - Adjacent free partitions combined
  - OS maintains information about:
    - allocated partitions
    - free partitions (holes)

6.14 Dynamic Storage-Allocation Problem

- How to satisfy a request of size nfrom a list of free holes?
- First-Fit: Allocate first hole that is big enough
- Best-Fit: Allocate smallest hole that is big enough; must search entire list, unless ordered by size
  - Produces smallest leftover hole
- Worst-Fit: Allocate largest hole; must also search entire list
  - Produces largest leftover hole
- First-fit and best-fit better than worst-fit in terms of speed and storage utilization

6.15 Fragmentation

- External Fragmentation: Total memory space exists to satisfy a request, but it is not contiguous
- Internal Fragmentation: Allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- First fit analysis reveals that given N blocks allocated, 0.5 N blocks lost to fragmentation
  - 1/3 may be unusable → 50-percent rule
- Solutions to External Fragmentation
  - Compaction
    - Shuffle memory contents to place all free memory together in one large block
    - E.g., move all used blocks to one end of memory
    - Compaction is possible onlyif relocation is dynamic, and is done at execution time
  - Non-contiguous memory allocation scheme
    - Segmentation
    - Paging
- Backing store same Fragmentation Problem

6.16 Segmentation

- Memory-Management Scheme that Supports User View of Memory
  - Maps programmer's view to physical memory
- A Program is a Collection of Segments
  - A Segment is a Logical Unit such as:
    - main program
    - procedure
    - function
    - method
    - object
    - local variables
    - global variables
    - common block
    - stack
    - symbol table
    - arrays
  - The Physical equivalents are:
    - Code
    - Global variables
    - Heap
    - Stack
    - Standard C library
-

6.17 Segmentation Architecture

- Logical Address consists of a Two-part Tuple: $(\text{segment-number}, \text{offset})$
- Segment table maps two-dimensional physical addresses; each table entry has:
  - base: contains starting physical address where segments reside in memory
  - limit: specifies length of segment
- Segment-table base register (STBR) points to segment table's location in memory
- Segment-table length register (STLR) indicates number of segments used by a program
  - segment number s is legal if $s < \text{STLR}$
- Protection
  - With each entry in segment table associate:
    - validation bit is $0 \rightarrow$ illegal segment
    - read/write/execute privileges
- Protection Bits Associated with Segments
  - Code sharing occurs at segment level
- Segments Vary in Length
  - Memory Allocation is a Dynamic Storage-Allocation Problem

6.18 Segmentation vs. Variable-Sized Contiguous Allocation

- Variable-Sized Contiguous Allocation
  - Needs to bring entire process into memory (code, data, stack, ...)
- Segmentation
  - Break program into different segments
  - Brings segments to memory on demand
  - No need to bring unused library methods or unused code & data segments to memory

6.19 Problems with Segmentation

- Code segment: 10MB
- Data segment: 100MB
- Stack segment: 20MB
- Library segment: 10MB
- Still significant amount of fragmentation
- Both external & internal fragmentation

6.20 Paging

- Physical Address Space of a Process can be Non-Contiguous
- Process is allocated physical memory whenever latter is available
- Avoids external fragmentation
- Avoids problem of varying sized memory chunks
- Divide Physical Memory into Fixed-sized blocks called Frames
  - Size is power of 2, between 512 bytes and 16 Mbytes

- Divide Logical Memory into Blocks of Same Size called Pages
- Keep Track of all Free Frames
- To Run a Program of size N pages, Need to Find N Free Frames and Load Program
- Set up a Page Table to Translate Logical to Physical Addresses
- Backing Store Likewise Split into Pages
- Still have Internal Fragmentation
- Calculating Internal Fragmentation
  - Page size = 2,048 bytes
  - Process size = 72,766 bytes
  - 35 pages + 1,086 bytes
  - Internal fragmentation of 2,048-1,086 = 962B
- So small frame sizes are desirable
- But each page table entry takes memory to track
- Page sizes growing over time
- Process View and Physical Memory now Very Different
- By Implementation, Process can only Access its Own Memory

6.21 Address Translation Scheme

- Address Generated by CPU divided into:
  - Logical address space: 2^m
  - Page size: 2^n
  - Page number ($m - n$ bits): used as an index into a page table which contains base address of each page in physical memory
  - Page offset (n bits): combined with base address to define physical memory address that is sent to memory unit

6.22 Implementation of Page Table

- Page Table Kept in Main Memory
- Page-Table Base Register (PTBR) Points to Page Table
- Page-Table Length Register (PTLR) Indicates Size of Page Table
- In this Scheme every Data/Instruction Access Requires two Memory Accesses
  - One for page table and one for data/instruction
- Two Memory Access Problem can be Solved by Use of a Special Fast-Lookup HW cache
  - Called Associative Memory or Translation Look-Aside Buffers (TLBs)
- Fully Associative TLB
  - Fast
  - Very costly
  - Typically 64 1024 entries
- TLBs work similar to Caches
  - Consists of two parts: Key (Tag) and a Value
- TLB Functionality

    - An incoming item compared with all keys
    - If found, corresponding value is returned
    - If not found → TLB miss
- Some TLBs Store Address-Space Identifiers (ASIDs) in each TLB entry
    - Uniquely identifies each process to provide address-space protection for that process
    - Otherwise need to flush at every context switch
- On a TLB Miss, Value Loaded into TLB for Faster Access Next Time
    - Replacement policies must be considered
    - Some entries wired down for permanent fast access (e.g., kernel code pages)

6.23 Associative Memory

- Associative Memory: Parallel Search
- Address Translation (p, d)
    - If p is in associative register, get frame number out
    - Otherwise get frame number from page table in memory

6.24 Effective Access Time

- Associative Lookup time unit = ϵ
- Hit ratio = α
- Memory Access Time = β
- Effective Access Time (EAT)
    - EAT = $(\beta + \epsilon)\alpha + (2 \times \beta + \epsilon)(1 - \alpha)$

6.25 Memory Protection

- Memory Protection Implemented by Associating Protection bit with each Frame to Indicate if Read-Only or Read-Write Access is Allowed
    - Can also add more bits to indicate page execute-only
- Valid-Invalid bit Attached to each Entry in Page Table
    - valid indicates that associated page is in process' logical address space, and is thus a legal page
    - invalid indicates that page is not in process' logical address space
    - Or use page-table length register (PTLR)
- Any Violations Result in a Trap to Kernel

6.26 Shared Pages

- Shared code
    - One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems)
    - Similar to multiple threads sharing same process space
    - Also useful for inter-process communication if sharing of read-write pages is allowed
- Private Code and Data
    - Each process keeps a separate copy of code and data
    - Pages for private code and data can appear anywhere in logical address space

6.27 Structure of the Page Table

- Memory Structures for Paging can Get Huge using Straight-Forward Methods
- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

6.28 Hierarchical Page Tables

- Break up Logical Address Space into Multiple Page Tables
- A Simple Technique is a Two-Level Page table
- We then Page the page table
- Two-Level Paging Example
  - A Logical Address (on 32-bit machine with 1K page size) Divided into:
    - A page number consisting of 22 bits
    - A page offset consisting of 10 bits
  - Since Page Table is paged, page Number is further Divided into:
    - A 12-bit page number
    - A 10-bit page offset
  - Thus, a Logical Address is as follows:
    - p_1 is an index into outer page table
    - p_2 is displacement within page of inner page table
  - Known as forward-mapped page table
- Even two-level paging scheme not Sufficient
- Three-Level Paging Scheme

6.29 Hashed Page Tables

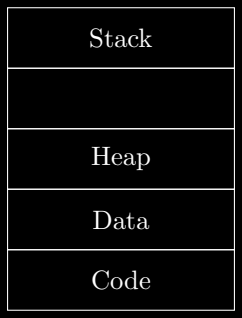
- Common in Address Spaces greater than 32 bits
- Virtual Page Number Hashed into a Page Table
  - This page table contains a chain of elements hashing to same location
- Each element contains (1) virtual page number (2) value of mapped page frame (3) a pointer to next element
- Virtual page numbers are compared in this chain searching for a match
  - If a match is found, corresponding physical frame is extracted

6.30 Inverted Page Table

- One Entry for each Real Page of Memory
  - Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- Entry consists of virtual address of page stored in that real memory location, with information about process that owns that page
- Pros
  - Decreases memory needed to store each page table
- Cons
  - Increases time needed to search table when a page reference occurs
  - Stored by physical addresses
- Use Hash Table to Limit Search to one or at most a few page-table entries
  - TLB can accelerate access

7 Virtual Memory

7.1 Virtual-Address Space



Virtual-Address Space Layout

-
- Usually design Logical Address Space for Stack to start at Max Logical Address and grow down while heap grows up
- Maximizes address space use
- Unused address space between two is hole
- No physical memory needed until heap or stack grows to a given new page
- Enables sparse address spaces with holes left for growth, dynamically linked libraries, etc
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during `fork()`, speeding process creation

7.2 Demand Paging

- Could Bring Entire Process into Memory at Load Time
- Or bring a Page into Memory only when it is Needed
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
- Similar to paging system with swapping
- Page is needed → reference to it
  - Invalid reference → abort
  - not-in-memory → Bring to memory
- Lazy Swapper: never swaps a page into memory unless page will be needed
  - Swapper that deals with pages is a pager

7.3 Basic Concepts

- With swapping, Pager Guesses which pages will be used before swapping out again
  - Instead, pager brings in only those pages into memory
- How to determine that set of pages?
  - Need new MMU functionality to implement demand paging
- If pages needed are already memory resident
  - No difference from non demand-paging
- If page needed and not memory resident
  - Need to detect and load page into memory from storage
  - Without changing program behavior
  - Without programmer needing to change code

7.4 Valid-Invalid Bit

- With each page table entry a valid-invalid bit is associated
  - in-memory: memory resident (1)
  - not-in-memory: not memory resident (0)
- Initially valid-invalid bit is set to 0 on all entries

7.5 Page Fault

1. If there is a reference to a page, 1st reference to that page will trap to OS: Page Fault
2. OS looks at table to decide:
   - Invalid reference → abort
   - Just not-in-memory → go to step 3
3. Find free frame
4. Swap page into frame via scheduled disk operation
5. Reset tables to indicate page now in memory (Set validation bit = 1)
6. Restart the instruction that caused the page fault

7.6 Aspects of Demand Paging

- Extreme case: start process with no pages in memory
  - OS sets instruction pointer to first instruction of process, non-memory-resident → page fault
  - For every other process pages on first access
  - Pure demand paging
- Can an instruction cause multiple page faults?
  - Consider fetch and decode of instruction which adds two numbers from memory and stores result back to memory → significant performance drop
  - Not a real case due to locality of reference
- HW support needed for Demand Paging
  - Page table with valid / invalid bit
  - Secondary memory (swap device with swap space)
  - Instruction restart

7.7 Instruction Restart

- Consider an instruction that could access several different locations
  - Block move
  - Auto increment/decrement location
  - Restart the whole operation
    - What if source and destination overlap?

7.8 Performance of Demand Paging

- Stages in Demand Paging (worse case)
  1. Trap to OS
  2. Save user registers and process state
  3. Determine that the interrupt was a page fault
  4. Check that page reference was legal and determine location of page on disk
  5. Issue a read from disk to a free frame:
     - Wait in a queue for this device until read request is serviced
     - Wait for device seek and/or latency time
     - Begin transfer of the page to a free frame
  6. While waiting, allocate CPU to some other user
  7. Receive an interrupt from disk I/O subsystem (I/O completed)
  8. Save registers and process state for other user
  9. Determine that interrupt was from disk
  10. Correct page table and other tables to show page is now in memory
  11. Wait for CPU to be allocated to this process again
  12. Restore user registers, process state, and new page table, then resume interrupted instruction
- Three Major Activities
  - Service interrupt: careful coding means just several hundred instructions needed
  - Read page: lots of time
  - Restart process: again just a small amount of time
- Page Fault Rate: $0 \leq p \leq 1$
  - if $p = 0$ no page faults
  - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)
  - EAT $= (1 - p) \times$ memory access time $+ p \times$ (page fault overhead + swap page out + swap page in)

7.9 Copy-on-Write

- Copy-on-Write (COW) Allows both parent and child processes to initially share same pages in memory
  - If either process modifies a shared page, only then is the page copied
- COW allows more efficient process creation
  - As only modified pages are copied
- In general, free pages are allocated from a pool of zero-fill-on-demand pages
  - Pool should always have free frames for fast demand page execution
  - Don't want to have to free a frame as well as other processing on page fault
- `vfork()`
  - Virtual memory fork system call
  - Has parent suspend and child using copy-on-write address space of parent

7.10 What Happens if There is no Free Frame?

- Used up by Process Pages
- Page Replacement
  - Find some page in memory, but not really in use, page it out
  - Algorithm
  - Performance: want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

7.11 Page Replacement

- Prevent Over-allocation of memory
  - By controlling degree of multi-programming
  - By modifying page-fault service routine to include page replacement
- Use Modify (Dirty) bit to reduce overhead of page transfers
  - Only modified pages are written to disk
- Page Replacement completes Separation between Logical memory and Physical memory
  - Large virtual memory can be provided on a smaller physical memory

7.12 Basic Page Replacement

1. Find Location of Desired Page on Disk
2. Find a Free Frame:
   - If there is a free frame, use it
   - If there is no free frame, use a page replacement algorithm to select a victim frame
   - Write victim frame to disk if dirty
3. Bring Desired Page into (newly) free frame; update page and frame tables
4. Continue Process by Restarting Instruction that caused trap

7.13 Page and Frame Replacement Algorithms

- Frame-Allocation Algorithm determines
  - How many frames to give each process?
  - Which frames to replace?
- Page-Replacement Algorithm
  - Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing number of page faults on that string
  - String is just page numbers, not full addresses
  - Repeated access to same page does not cause a page fault
  - Results depend on number of frames available

7.14 First-In-First-Out (FIFO) Algorithm

- Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
- 3 frames (3 pages can be in memory at a time per process)
- How to track ages of pages?
  - Just use a FIFO queue
- Belady's Anomaly
  - Adding more frames can cause more page faults!

7.15 Optimal Algorithm

- Replace Page that will not be used for Longest Period of Time (in future)
- How do you know this?
  - Can't read the future
- Used just as a Reference Model
  - For measuring how well your algorithm performs

7.16 Least Recently Used (LRU) Algorithm

- Use Past Knowledge rather than Future
- Replace Page that has not been Used in Most Amount of Time
- Associate Time of Last Use with each page
- Generally good algorithm and frequently used
- Counter Implementation
  - Every page entry has a counter
  - Every time page is referenced through this entry, copy the clock into the counter
  - When a page needs to be changed, look at the counters to find smallest value
    - Search through table needed
  - Cons
    - Write to memory on every memory access
    - Needs to look into all counters for smallest value
    - Counter overflow is an issue
- Stack Implementation
  - Keep stack of page numbers in a double link-list
  - Page referenced
    - Move it to top
    - Requires up to 6 pointers to be changed
  - But each update more expensive
  - Pros
    - No search for replacement
- Stack Algorithms
  - An algorithm for which it can be shown that set of pages in memory for n frames is always a subset of set of pages that would be in memory with n+1 frames
  - Informally: Don't have Belady's anomaly
  - Example of Stack Algorithms
    - LRU
    - OPT

7.17 LRU Approximation Algorithms

- LRU needs Special HW and still slow
- Reference Bit
  - With each page associate a bit, initially = 0
  - When page is referenced, bit set to 1
  - Replace any with reference bit = 0 (if one exists)
  - We do not know the order, however
- Additional-Reference-Bits Algorithm
  - Use multiple history bits
  - OS shifts reference bits at intervals
- Second-Chance Algorithm
  - Generally FIFO, plus HW-provided reference bit
  - Clock replacement
  - If page to be replaced has
    - Reference bit = 0 → replace it
    - Reference bit = 1 then:
      - Set reference bit 0, leave page in memory
      - Replace next page, subject to same rules

7.18 Enhanced Second-Chance Algorithm

- Using Reference Bit and Modify Bit
  - Take ordered pair (reference, modify)
    1. (0, 0) neither recently used not modified. best candid
    2. (0, 1) not recently used but modified. not quite as good, must write out before replacement
    3. (1, 0) recently used but clean. probably will be used again soon
    4. (1, 1) recently used and modified. probably will be used again soon and need to write out before replacement
- Use Same Scheme as Clock
  - First search in class 1, then 2 and 3 and lastly 4
  - Favors modified pages to reduce number of I/Os
  - Might need to search circular queue several times

7.19 Counting Algorithms

- Keep a counter of number of references that have been made to each page
- Least Frequently Used (LFU) Algorithm
  - Replaces page with smallest count
  - Typically counter shifted to right at regular intervals to avoid accumulated references
- Most Frequently Used (MFU) Algorithm
  - Based on argument that page with smallest count probably just brought in and has yet to be used
- Issues with LFU & MFU → Not common
  - Expensive to implement
  - Do not approximate OPT replacement well

7.20 Page-Buffering Algorithms: Enhanced Techniques

- Keep a Pool of Free Frames
  - Frame available when needed, not found at fault time
  - Read page into free frame and select victim to evict and add to free pool
  - When convenient, evict victim
- Keep List of Modified Pages
  - When backing store otherwise idle, write pages there and set to non-dirty → faster eviction
- Keep Free Frame Contents Intact and Note What is in Them
  - If referenced again before reused, no need to load contents again from disk
  - Generally useful to reduce penalty if wrong victim frame selected

7.21 Applications and Page Replacement

- All of these algorithms have OS Guessing about Future Page Access
- Some Applications have Better Knowledge (e.g., databases)
- Memory Intensive Applications can cause Double Buffering
  - OS keeps copy of page in memory as I/O buffer
  - Application keeps page in memory for its own work
- OS can Give Direct Access to Disk
  - Array called raw disk mode
  - I/O accesses to raw disk called raw I/O
  - Bypasses all file-system services such as demand paging, buffering, locking, and prefetching

7.22 Allocation of Frames

- How to allocate fixed amount of free memory among various processes?
- Cannot allocate more than total number of available frames
  - Unless there is page sharing
- Must allocate minimum number of frames
  - Significant page fault rate → degraded performance
- Number of allocated frames for a process decreases → page fault rate increases
- Upon a page fault → instruction must be restarted
- Must have enough frames to hold all different pages that any single instruction can reference
- Worst Case Scenario
  - When an ISA allows multiple levels of indirection
  - E.g., 100 levels of indirection → would need 101 pages in physical memory
  - Solution: put a limit on maximum number of indirection
- Minimum number of Frames per Process
  - Depends on ISA and locality of references
- Maximum number of Frames per Process
  - Available physical memory & number of processes in system
- Two Major Allocation Schemes
  - Fixed allocation & priority allocation

7.23 Fixed Allocation

- Equal Allocation
  - E.g., if there are 100 frames (after allocating frames for OS) and 5 processes, give each process 20 frames
  - Keep some as free frame buffer pool
- Proportional Allocation
  - Allocate according to size of process
  - Dynamic as degree of multiprogramming, process sizes change

7.24 Priority Allocation

- Use a Proportional Allocation Scheme using Priorities rather than Size
- If Process P_i Generates a Page Fault
  - Select for replacement one of its frames
  - Select for replacement a frame from a process with lower priority number

7.25 Global vs. Local Allocation

- Global Replacement: process selects a replacement frame from set of all frames; one process can take a frame from another
  - But then process execution time can vary greatly
  - But greater throughput so more common
- Local Replacement: each process selects from only its own set of allocated frames
  - More consistent per-process performance
  - But possibly underutilized memory

7.26 Non-Uniform Memory Access

- So far all memory accessed equally
- Many systems are NUMA, speed of access to memory varies
  - Consider system boards containing CPUs and memory, interconnected over a system bus
- Optimal performance comes from allocating memory close to CPU on which thread scheduled
  - And modifying scheduler to schedule thread on same system board when possible
  - Solved by Solaris by creating lgroups
    - Structure to track CPU / Memory low latency groups
    - Used my schedule and pager
    - When possible schedule all threads of a process and allocate all memory for that process within the lgroup

7.27 Thrashing

- If a Process does not have Enough Pages, Page-Fault Rate is Very High
  - Page fault to get page
  - Replace existing frame
  - But quickly need replaced frame back
  - This leads to:
    - Low CPU utilization
    - OS thinking that it needs to increase degree of multiprogramming
    - Another process added to system
- Thrashing: Process is Busy Swapping Pages in and out

7.28 Demand Paging and Thrashing

- Why does Demand Paging work?
- Locality Model
  - Process migrates from one locality to another
  - Localities may overlap
- Why does Thrashing Occur?
- ΣSize of Locality > Total Memory Size
  - Limit effects by using local or priority page replacement

7.29 Working-Set Model

- Δ is working-set window; a fixed number of page references
- WSS_i (working set of Process P_i) = total number of pages referenced in most recent Δ (varies in time)
  - If Δ too small $\rightarrow$ will not encompass entire locality
  - If Δ too large $\rightarrow$ will encompass several localities
  - If $\Delta = \infty \rightarrow$ will encompass entire program
- $\Delta = \Sigma WSS_i \rightarrow$ Total Demand Frames
  - Approximation of locality
- If $\Delta >$ Total Memory $\rightarrow$ Thrashing
- Policy: if thrashing, then suspend or swap out one of processes

7.30 Keeping Track of Working Set

- Approximate with Interval Timer and a Reference Bit
- Example: $\Delta = 10000$
  - Timer interrupts after every 5000 time units
  - Keep in memory 2 bits for each page
  - Whenever a timer interrupts copy and sets values of all reference bits to 0
  - If one of bits in memory = 1 $\rightarrow$ page in working set
- Improvement=10 bits & interrupt every 1000 time units

7.31 Page-Fault Frequency

- More Direct Approach than WSS
- Establish acceptable Page-Fault Frequency (PFF) Rate and Use Local Replacement Policy
  - If actual rate too low $\rightarrow$ process loses frame
  - If actual rate too high $\rightarrow$ process gains frame

7.32 Working Sets and Page Fault Rates

- Direct Relationship between Working Set of a Process and its Page-Fault Rate
- Working Set Changes over Time
- Peaks and Valleys over Time

7.33 Memory-Mapped Files

- Memory-Mapped File I/O allows File I/O to be treated as routine memory access by mapping a disk block to a page in memory
- A File is initially Read using Demand Paging
  - A page-sized portion of file is read from file system into a physical page
  - Subsequent reads/writes to/from file are treated as ordinary memory accesses
  - Simplifies and Speeds File Access by Driving File I/O through memory rather than `read()` and `write()` system calls
- Also Allows Several Processes to Map same File allowing Pages in Memory to be Shared
- But when does Written Data Make it to Disk?
  - Periodically and/or at file `close()` time
  - For example, when pager scans for dirty pages

7.34 Memory-Mapped File Technique for all I/O

- Some OSes uses memory mapped files for standard I/O
- Process can explicitly request memory mapping a file via `mmap()` system call
  - Now file mapped into process address space
- Solaris: For standard I/O (`open()`, `read()`, `write()`, `close()`), mmap anyway
  - But map file into kernel address space
  - Process still does `read()` and `write()`
    - Copies data to and from kernel space and user space
  - Uses efficient memory management subsystem
    - Avoids needing separate subsystem
- COW can be used for read/write non-shared pages
- Memory mapped files can be used for shared memory (although again via separate system calls)

7.35 Allocating Kernel Memory

- Kernel Memory Treated Differently from User Memory
- Often Allocated from a Free-Memory Pool
  - Kernel requests memory for structures of varying sizes
  - Some kernel memory needs to be contiguous
    - Structures for device I/O
    - Page tables

7.36 Buddy System

- Allocates Memory from Fixed-Size Segment consisting of Physically-Contiguous Pages
- Memory allocated using power-of-2 allocator
  - Satisfies requests in units sized as power of 2
  - Request rounded up to next highest power of 2
  - When smaller allocation needed than is available, current chunk split into two buddies of next-lower power of 2
  - Continue until appropriate sized chunk available
- Advantage: quickly coalesce unused chunks into larger chunk
- Disadvantage: fragmentation

7.37 Slab Allocator

- Alternate Strategy
- Slab is one or more physically contiguous pages
- Cache consists of one or more slabs
- Single cache for each unique kernel data structure
  - A cache for file objects, a cache for semaphores
  - Each cache filled with objects: instantiations of data structure
- When cache created, filled with objects marked as free
- When structures stored, objects marked as used
- If slab is full of used objects, next object allocated from empty slab
  - If no empty slabs, new slab allocated
- Benefits include no fragmentation, fast memory request satisfaction

8 Process Synchronization

8.1 Critical Section Problem

- Consider System of n Processes $\{p_0, \ldots, p_{n-1}\}$
- Each Process has Critical Section Segment of Code
  - Process may be changing common variables, updating table, writing file, etc
  - When one process in critical section, no other may be in its critical section
- Critical Section Problem: Design a Protocol so that Processes can cooperate
  - Each process must ask permission to enter critical section in entry section, may follow critical section with exit section, then remainder section
- General Structure of Process p_i

```
do
{
    /* entry section */
    /* critical section */
    /* exit section */
    /* remainder section */
} while (true);
```

- Algorithm for Process p_i

```
do
{
    while (turn == j)
        ;
    critical section
        turn = j;
    remainder section
} while (true);
```

8.2 Solution to Critical-Section Problem

- Critical-Section Problem Must Satisfy Following Requirements
  - Mutual exclusion
    - If process p_i is executing in its critical section, then no other processes can be executing in their critical sections
  - Progress
    - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then selection of processes that will enter critical section next cannot be postponed indefinitely
  - Bounded waiting
    - A bound must exist on number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
- Assume that each process executes at a nonzero speed
- No assumption concerning relative speed of n processes

8.3 Race Conditions in OS Kernel

- Example 1
  - A kernel data structure maintaining a list of all open files
  - Two processes open files to simultaneously, separate update to the list $\rightarrow$ RC
- Example 2

- Updating structures maintaining memory allocation
- Example 3
  - Updating list of processes

8.4 Critical-Section Handling in OS

- Two approaches for handling CS depending on if kernel is preemptive or non-preemptive
  - Preemptive: allows preemption of process when running in kernel mode
  - Non-preemptive: runs until it exits kernel mode, blocks, or voluntarily yields CPU
    - Does not allow a process running in kernel mode to be preempted
    - Essentially free of race conditions in kernel mode
    - Only one process is active in kernel at a time

8.5 Preemptive vs. Non-Preemptive Kernels

- Preemptive Kernels
  - More suitable for real-time programming
  - More responsive
  - Very difficult to design for SMP architectures
    - Two kernel-mode processes run simultaneously on different cores
- Non-Preemptive Kernels
  - Can run for an arbitrary long period before relinquishing CPU
  - No race condition in kernel mode

8.6 Peterson's Solution

- Good Algorithmic Description of Solving Problem (a SW solution)
- Two-Process Solution
- Assume that `load()` and `store()` Machine-Language Instructions are Atomic
- Two Processes Share Two Variables:

```
int turn;
Boolean flag[2];
```

  - Variable turn
    - Indicates whose turn it is to enter critical section
    - turn i or j
  - Flag Array
    - Used to indicate if a process is ready to enter critical section
    - `flag[i]` is true implies that process p_i is ready to enter its critical section
- Algorithm for Process p_i

```
do
{
    flag[i] = true;
    turn = j;
    while (flag[j] &&turn = = j)
        ;
    /* critical section */
    flag[i] = false;
    /* remainder section */
} while (true);
```

- Provable that three CS requirement are met:
  - Mutual exclusion is preserved p_i enters CS only if:
    - either `flag[j]` is false or `turn` is i
  - Progress requirement is satisfied (no deadlock)
  - Bounded-waiting requirement is met (no starvation)

8.7 Synchronization Hardware

- SW Solutions such as Peterson's Not Guaranteed to Work on Modern Architectures
- Many Systems Provide HW Support for Implementing Critical Section Code
- Any Solution to CS requires Locking
  - All solutions hereafter based on locking Idea
  - Protecting critical regions via locks
- Locking Solves CS Problem
  - Process must acquire a lock before entering a CS
  - Process releases lock when it exists CS
- Uniprocessors: could Disable Interrupts
  - Currently running code would execute without preemption
  - Generally too inefficient on multiprocessor systems
  - Too time-consuming
- Modern Machines Provide Special Atomic HW Instructions
  - Either test memory word and set value
  - Or swap contents of two memory words

8.8 Solution to CS Problem using Locks

```
do
{
    /* acquire lock */
    /* critical section */
    /* release lock */
    /* remainder section */
} while (true);
```

8.9 test_and_set Instruction

```
Boolean test_and_set(Boolean *target)
{
    Boolean rv = *target;
    *target = true;
    return rv:
}
```

- Executed atomically
- Returns original value of passed parameter
- Set new value of passed parameter to true
- Shared Boolean Variable Lock, Initialized to false

```
do
{
    while (test_and_set(&lock))
        ; /* do nothing */
    /* critical section */
    lock = false;
    /* remainder section */
} while (true);
```

8.10 compare_and_swap Instruction

```
int compare_and_swap(int *value, int expected, int new_value)
{
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

- Executed Atomically
- Returns Original Value of Passed Parameter value
- Set Variable value the value of passed parameter new_value but only if value equals expected. That is, swap takes place only under this condition.
- Shared integer lock initialized to 0

```
do
{
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */
    /* critical section */
    lock = 0;
    /* remainder section */
} while (true);
```

8.11 Bounded-waiting Mutual Exclusion with test_and_set

```
do
{
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false;
    else
        waiting[j] = false;
    /* remainder section */
} while (true);
```

8.12 Mutex Locks

- HW Solutions Complicated and generally Inaccessible or Invisible to Application Programmers
- OS Designers build SW Tools to Solve Critical Section Problem
- Simplest is mutex (Abbreviate for mutual exclusion) lock
- Protect a Critical Section by first `acquire()` a lock then `release()` the lock
    - Boolean variable indicates if lock is available or not

8.13 `acquire()` and `release()`

```
acquire()
{
    while (!available)
        ; /* busy wait */
    available = false;
}
```

```
release()
{
    available = true;
}
```

```
do
{
    /* acquire lock */
    /* critical section */
    /* release lock */
    /* remainder section */
} while (true);
```

- Calls to `acquire()` and `release()` must be Atomic
  - Usually implemented via HW atomic instructions
- But this Solution requires Busy Waiting
  - This lock therefore called a spinlock
  - Can degrades performance
  - When locks expected to be held for short times, spinlocks are useful (no context switch time)
  - More useful in multi-processors

8.14 Semaphore

- Synchronization Tool that Provides more Sophisticated Ways (than Mutex locks) for Process to Synchronize their Activities
- Semaphore S: Integer Variable
- Can only be Accessed via Two Indivisible (Atomic) Operations
  - `wait()` and `signal()`
- Originally called `P()` and `V()`
- Definition of `wait()` operation

```
wait(S)
{
    while (S <= 0)
        ; // busy wait
    S--;
}
```

- Definition of `signal()` operation

```
signal(S)
{
    S++;
}
```

8.15 Semaphore Usage

- Counting Semaphore
  - Integer value can range over an unrestricted domain
  - Used to control access to a given resource consists of a finite number of instances
  - Semaphore is initialized to number of available resources
- Binary Semaphore (Same as a mutex lock)
  - Can deal with CS problem for multiple processes

- Integer value can range only between 0 and 1
- Mutex is initialized to 1

- Semaphore can be Used for various Synch Problems
- Consider p_1 & p_2 that require s_1 to happen before s_2: Create a semaphore synch initialized to 0

```
p1:
    s1;
    signal(synch);
```

```
p2:
    wait(synch);
    s2;
```

8.16 Semaphore Implementation

- Must Guarantee that no Two Processes can Execute `wait()` and `signal()` on Same Semaphore at Same Time
- Implementation becomes Critical Section Problem
  - `wait` and `signal` code are placed in critical section
  - Could now have busy waiting in critical section implementation
  - But implementation code is short
  - Little busy waiting if critical section rarely occupied
- Applications may Spend lots of Time in Critical Sections and hence this is not a good solution

8.17 Semaphore Implementation with no Busy waiting

- With each Semaphore, there is an Associated Waiting Queue
- Each Entry in a Waiting Queue has two Data Items:
  - Value (of type integer)
  - Pointer to next record in list
- Two Operations:
  - Block: place process invoking operation on appropriate waiting queue
  - Wakeup: remove one of processes in waiting queue and place it in ready queue

```
typedef struct
{
    int value;
    struct process *list;
} semaphore;
```

```
wait(semaphore *s)
{
    s->value--;
    if (s->value < 0)
    {
        add pi to s->list;
        block();
    }
}
```

```
signal(semaphore *s)
{
    s->value++;
    if (s->value <= 0)
    {
        remove pi from s->list;
        wakeup(pi);
    }
}
```

8.18 Deadlock and Starvation

- Deadlock: Two or More Processes are Waiting Indefinitely for an Event that can be Caused by only one of Waiting Processes
- Let s and q be 2 Semaphores Initialized to 1
-

```
p0:
    wait(S);
    wait(Q);
    ...
    signal(S);
    signal(Q);
```

```
p1:
    wait(Q);
    wait(S);
    ...
    signal(Q);
    signal(S);
```

- Starvation: Indefinite Blocking
  - A process may never be removed from semaphore queue in which it is suspended
  - When we remove processes from the semaphore waiting listing in LIFO (Last-In First Out) order
- Priority Inversion
  - Scheduling problem when lower-priority process holds a lock needed by higher-priority process
- Example: Consider L, M, and H
  - Priorities $L < M < H$
  - H requires resource R
  - R is being used by L
  - M comes to ready queue $\rightarrow$ preempts process L
  - A lower priority (M) has affected how long H must wait for L to relinquish resource R
- Possible Solution to Priority Inversion
  - Only allow two levels of priorities
    - Insufficient for most general-purpose OSes
  - Priority-inheritance protocol
    - All processes that are accessing resources needed by a higher-priority process inherit the higher priority until they are finished with resource in question
    - Priorities $L < M < H$
      - Using priority-inheritance: $L = H$

8.19 Classical Problems of Synchronization

- Classical Problems used to Test Synchronization Schemes
  - Bounded-Buffer Problem
    - n buffers, each can hold one item
    - Semaphore mutex initialized to value 1
    - Semaphore full initialized to value 0
    - Semaphore empty initialized to value n

```
do
{
    ...
        /* produce an item in next_produced */
        ... wait(empty);
    wait(mutex);
    ...
        /* add next produced to the buffer */
        ... signal(mutex);
    signal(full);
} while (true);
```

```
do
{
    wait(full);
    wait(mutex);
    ...
        /*remove an item from buffer to
        next_consumed*/
        ... signal(mutex);
    signal(empty);
    ...
        /* consume the item in next consumed */
        ...
} while (true);
```

- Readers-Writers Problem
  - A Data Set is Shared among a Number of Concurrent Processes
    - Readers: only read data set; they do not perform any updates
    - Writers: can both read and write
  - Problem: Allow Multiple Readers to Read at Same Time
    - Only one single writer can access shared data at the same time
  - Several Variations of How Readers and Writers are Considered
    - All involve some form of priorities to readers or writers
  - Shared Data
    - Data set
    - Semaphore `rw_mutex` initialized to 1: Common to both reader and writer
    - Semaphore `mutex` initialized to 1: Used to ensure mutual exclusion when read_count is updated
    - Integer `read_count` initialized to 0: How many processes are currently reading the object

```
do
{
    wait(rw_mutex);
    ...
        /* writing is performed */
        ... signal(rw_mutex);
} while (true);
```

```
do
{
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);
    ...
        /* reading is performed */... wait(mutex);
    read count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

  - One Variation: no reader kept waiting unless writer has permission to use shared object
  - Another Variation: once writer is ready, it performs the write ASAP
  - Both may have starvation leading to even more variations

- Problem is solved on some systems by kernel providing reader-writer locks

- Dining-Philosophers Problem
  - Philosophers spend their lives alternating thinking & eating
  - Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
  - In case of 5 philosophers
    - Shared data: Bowl of rice (data set), Semaphore chopstick [5] initialized to 1

```
do
{
    wait(chopstick[i]);
    wait(chopStick[(i + 1) % 5]);
    //  eat
    signal(chopstick[i]);
    signal  (chopstick[(i + 1) % 5]);
    //  think
} while (TRUE);
```

  - Deadlock Handling
    - Allow at most 4 philosophers to be sitting simultaneously at table
    - Allow a philosopher to pick up the forks only if both are available
    - Use an asymmetric solution: An odd-numbered philosopher picks up first the left chopstick and then the right chopstick, even-numbered philosopher picks up first the right chopstick and then the left chopstick.

8.20 Problems with Semaphores

- Incorrect Use of Semaphore Operations:
  - `signal(mutex) ... wait(mutex)`
  - - Several processes maybe executing in their critical sections simultaneously, violating mutual-exclusion requirement
  - `wait(mutex) ... wait(mutex)`
    - Deadlock will occur
  - Omitting of `wait(mutex)` or `signal(mutex)` (or both)
    - Mutual exclusion violated or a deadlock will occur
- Main Reasons for Incorrect Use of Semaphores
  - Programming error
  - An uncooperative programmer

8.21 Monitors

- A High-Level Abstraction that Provides a Convenient and Less Error-Prone Mechanism for Process Synchronization
- Abstract data type, internal variables only accessible by code within the procedure
  - External procedures are not allowed to access variables (similar to object-oriented principles)
- A Process Enters Monitor by Invoking one of Its Procedures
- Only One Process may be Active within Monitor at a time (Typically handled by compilers)

```
monitor monitor_name
{
    Local Data
    Condition Variables
    procedure P1(...){...}
    procedure Pn(...){...}
    Initialization code(...){ ... }
}
```

8.22 Condition Variables

- `condition x, y;`
- Two Operations are Allowed on a Condition Variable:
- `x.wait()`: a process that invokes operation is suspended until `x.signal()`
- `x.signal()`: resumes one of processes (if any) that invoked `x.wait()`
    - If no `x.wait()` on the variable, then it has no effect on the variable

8.23 Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
{
    enum
    {
        THINKING; HUNGRY, EATING
    }
    state[5];
    condition self[5];

    void pickup(int i)
    {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait;
    }
    void putdown(int i)
    {
        state[i] = THINKING;
        // test left and right neighbors
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
    void test(int i)
    {
        if ((state[(i + 4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i + 1) % 5] != EATING))
        {
            state[i] = EATING;
            self[i].signal();
        }
    }
    initialization_code()
    {
        for (int i = 0; i < 5; i++)
            state[i] = THINKING;
    }
}
```

- Each philosopher i invokes operations `pickup()` and `putdown()`

```
DiningPhilosophers.pickup(i);
EAT
DiningPhilosophers.putdown(i);
```

- No deadlock, but starvation is possible

9 Deadlocks

9.1 System Model

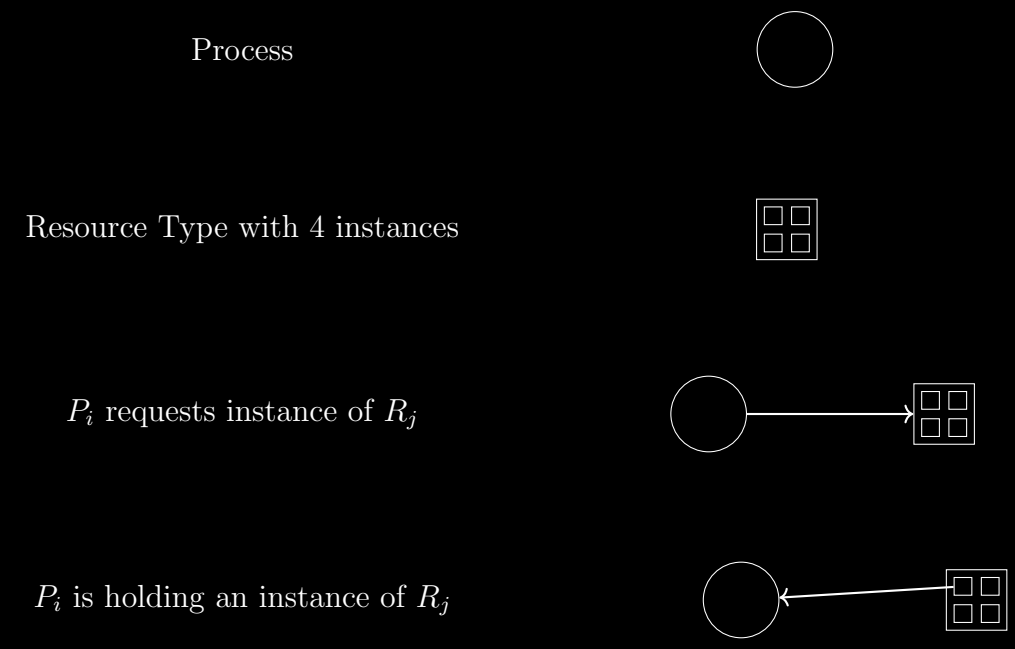
- System Consists of Resources
  - To be distributed among a number of competing processes
- Resource Types $R_1, R_2, \ldots, R_m$
- Each Resource Type R_i has W_i instances
- Each Process Utilizes a Resource as follows
  - Request, Use, Release
    - Accomplished using system calls such as `request()` and `release()` device or `open()` and `close()` file, or `allocate()` and `free()` memory
- Physical Resources
  - Printers, tape drives, memory space, or CPU cores/cycles
- Logical Resources
  - Files, semaphores, and monitors
- Multithreaded Programs:
  - Good candidate for deadlock
  - Multiple threads compete for shared resources

9.2 Deadlock Characterization

- Deadlock can arise if four conditions hold simultaneously
- Mutual Exclusion: only one process at a time can use a resource
- Hold and Wait: a process holding at least one resource is waiting to acquire additional resources held by other processes
- No Preemption: a resource can be released only voluntarily by process holding it, after that process has completed its task
- Circular Wait: there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, ..., P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.

9.3 Resource-Allocation Graph

- A set of vertices V and a set of edges E
- V is partitioned into two types:
  - $P = P_1, P_2, \ldots, P_n$, set consisting of all processes in the system
  - $R = R_1, R_2, \ldots, R_m$, set consisting of all resource types in the system
- E is partitioned into two types:
  - Request Edge: directed edge $P_i \to R_j$
  - Assignment Edge: directed edge $R_j \to P_i$



Resource Allocation Graph Symbols

-

9.4 Basic Facts

- If Graph Contains no Cycles → No Deadlock
- If Graph Contains a Cycle →
  - Only one instance per resource type → Deadlock
  - Cycle involves only a set of resource types, each of which has only a single instance → Deadlock
  - Several instances per resource type → Possibility of deadlock

9.5 Methods for Handling Deadlocks

- Ensure that System will Never Enter a Deadlock State
  - Deadlock prevention: try to violate one of necessary conditions for deadlock
  - Deadlock avoidance: try to regulate how/when requests can be made to acquire resources
    - More conservative approach than deadlock prevention
- Allow System to enter a Deadlock State and then recover
- Ignore Problem and Pretend that Deadlocks Never occur in system
  - Used by most OSes, including UNIX
  - Up to application developer to detect and handle deadlocks
- What if Deadlocks are not Resolved?
  - Deterioration of system performance
    - Eventually need a manual restart
  - Deadlock occur very infrequent → cheaper approach in mainstream applications
    - Instead of employing prevention, avoidance, or detection and recovery methods

9.6 Deadlock Prevention

- Mutual Exclusion
  - Not required for sharable resources
  - Only one process can access a non-sharable resource at a time
  - Must hold for non-sharable resources
  - Example: Read-only files
- Hold and Wait: must guarantee that whenever a process requests a resource, it does not hold any other resources
  - Solution 1: Require process to request and be allocated all its resources before it begins execution
  - Solution 2: Or allow process to request resources only when process has none allocated to it
  - Cons
    - Low resource utilization
    - Starvation possible
- No Preemption
  - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
  - Preempted resources are added to list of resources for which the process is waiting
  - Process will be restarted only when it can regain its old resources, as well as new ones
  - This protocol applicable only to resources whose state can be easily saved and restored later
    - CPU registers and memory space: applicable
    - Printers and tape drives: not (easily) applicable
- Circular Wait: impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration
  - A process which holds $R(i)$, can request instance of $R(j)$ if $F(Rj) > F(Ri)$
  - Ensuring order by application developer
  - Can use lock-order verifier (e.g., witness in FreeBSD)

9.7 Deadlock Example with Lock Ordering

- Lock Ordering does not Guarantee Deadlock Prevention if Locks can be acquired Dynamically
  - Ordering is broken with unordered arguments

```
void transaction(Account from, Account to, double amount)
{
    mutex lock1, lock2;
    lock1 = get_lock(from);
    lock2 = get_lock(to);
    acquire(lock1);
    acquire(lock2);
    withdraw(from, amount);
    deposit(to, amount);
    release(lock2);
    release(lock1);
}
```

- Transactions 1 and 2 execute concurrently
- Transaction 1 transfers $25 from account A to account B, & Transaction 2 transfers $50 from account B to account A

9.8 Deadlock Avoidance

- Requires that system has some additional a priori information available
- Simplest and most useful model requires that each process declare maximum number of resources of each type that it may need
- Deadlock-avoidance algorithm dynamically examines resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation state is defined by number of available and allocated resources, and maximum demands of processes

9.9 Safe State

- When a process requests an available resource, system must decide if immediate allocation leaves system in a safe state
- System is in safe state if there exists a sequence $(P_1, P_2, \ldots, P_n)$ (aka, safe sequence) of all processes in systems such that for each P_i, resources that P_i can still request can be satisfied by currently available resources and resources held by all P_j, with $j < i$
- If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
- When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
- When P_i terminates, P_{i+1} can obtain its needed resources, and so on

9.10 Basic Facts

- If a System is in Safe State $\rightarrow$ No Deadlocks
- If a System is in Unsafe State $\rightarrow$ Possibility of Deadlock
- Avoidance $\rightarrow$ Ensure that a System will Never enter an Unsafe State

9.11 Avoidance Algorithms

- Single Instance of a Resource Type
  - Use a resource-allocation graph
- Multiple Instances of a Resource Type
  - Use banker's algorithm

9.12 Resource-Allocation Graph Scheme

- Claim edge
  - $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j (represented by a dashed line)
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when resource is allocated to process
- Resource is Released by a process $\rightarrow$ Assignment Edge reconverts to a claim edge
- Resources must be claimed a priori in system

9.13 Resource-Allocation Graph Algorithm

- Suppose that P_i requests a R_j
- Request can be Granted only if Converting Request Edge to an Assignment Edge does not Result in Formation of a Cycle in Resource Allocation Graph
- Algorithm Complexity $O(n^2)$
- Low Resource Utilization
  - Resources might be available and not be allocated to processes

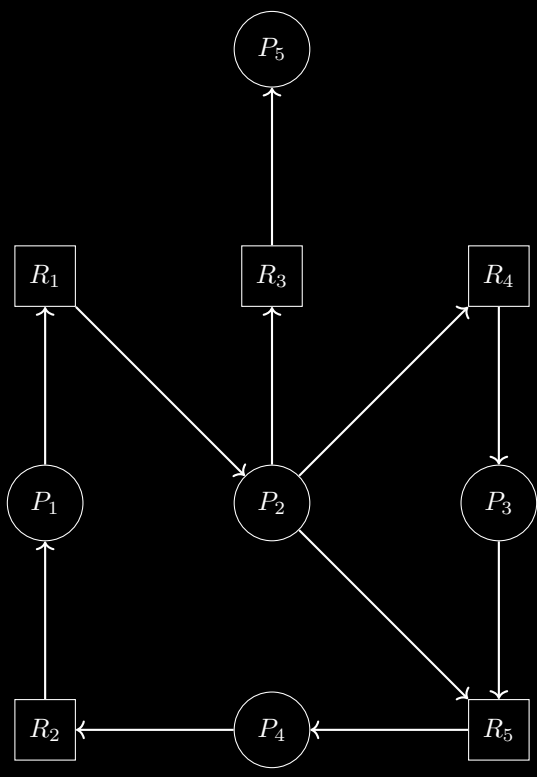
9.14 Deadlock Detection

- Allow System to Enter Deadlock State
- Detection Algorithm
  - Single instance of each resource type
  - Multiple instances of a resource type
- Recovery Scheme

9.15 Single Instance of Each Resource Type

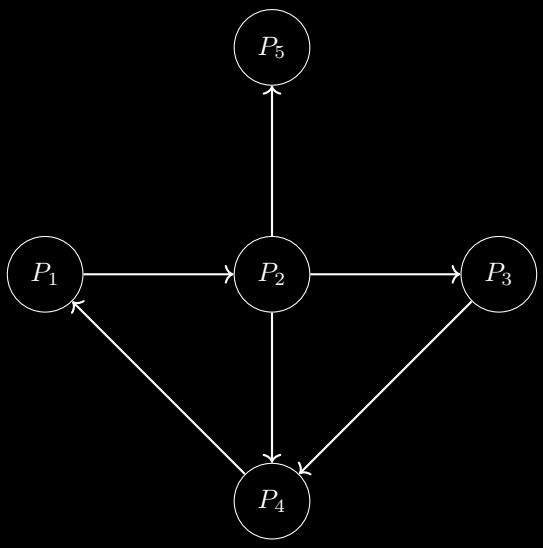
- Maintain wait-for graph
  - Nodes are processes
  - $P_i \rightarrow P_j$ if P_i is waiting for P_j
- Periodically invoke an Algorithm that Searches for a Cycle in graph
  - If there is a cycle $\rightarrow$ there exists a deadlock
- An Algorithm to Detect a Cycle in a Graph Requires an Order of n^2 operations
  - Where n is number of vertices in graph

9.16 Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph

-



Corresponding Wait-For Graph

-

9.17 Detection-Algorithm Usage

- When, and How often, to Invoke depends on:
  - How often a deadlock is likely to occur?
  - How many processes will need to be rolled back?
    - One for each disjoint cycle
- If Detection Algorithm is Invoked Arbitrarily, there may be many cycles in resource graph and so we would not be able to tell which of many deadlocked processes caused deadlock

9.18 Recovery from Deadlock: Process Termination

- Abort all Deadlocked Processes
  - Significant expense
  - Most processes need to be restarted for re-computation
- Abort one process at a time until Deadlock Cycle is Eliminated
  - After each process is aborted, a deadlock-detection algorithm must be invoked
- Issues with Aborting a Process
  - Updating a file
  - Printing data
- In which Order should we Choose to Abort?
  - Priority of process
  - How long process has computed, and how much longer to completion
  - Resources process has used
  - Resources process needs to complete
  - How many processes will need to be terminated
  - Is process interactive or batch?

9.19 Recovery from Deadlock: Resource Preemption

- Selecting a Victim: minimize cost
- Rollback: return to some safe state, restart process for that state
- Starvation: same process may always be picked as victim, include number of rollback in cost factor

10 File System

10.1 File Concept

- Contiguous Logical Address Space
- Types:
  - Data
    - Numeric
    - Character
    - Binary
  - Program
- Contents Defined by File's Creator
  - Many types: text file, source file, executable file

10.2 File Attributes

- Name: only information kept in human-readable form
- Identifier: unique tag (number) identifies file within file system
- Type: needed for systems that support different types
- Location: pointer to file location on device
- Size: current file size
- Protection: controls who can do reading, writing, executing
- Time, Date, and User Identification
  - Data for protection, security, and usage monitoring
- Information about Files Kept in Directory Structure
  - Maintained on disk
- Many Variations
  - Including extended file attributes such as file checksum
- Information Kept in Directory Structure

10.3 File Operations

- File is an Abstract Data Type
- `create`
- `write`: at Write Pointer Location
- `read`: at Read Pointer Location
- `seek`: reposition within File
- `delete`
- `truncate`
- `open(`F_i`)`: search directory structure on disk for entry F_i, and move content of entry to memory
- `close(`F_i`)`: move content of entry F_i in memory to directory structure on disk

10.4 Open Files

- Several Pieces of Data Needed to Manage Open Files:
  - Open-file table: tracks open files
  - File pointer: pointer to last read/write location, per process that has the file open
  - File-open count: counter of number of times a file is open to allow removal of data from open-file table when last processes closes it
  - Disk location of file: Info to locate file on disk kept in memory
  - Access rights: per-process access mode information

10.5 Open File Locking

- Provided by some OSs and File Systems
  - Similar to reader-writer locks
  - Shared lock similar to reader lock: several processes can acquire concurrently
  - Exclusive lock similar to writer lock
- Mediates Access to a File
- Mandatory or Advisory:
  - Mandatory: access is denied depending on locks held and requested
  - Advisory: processes can find status of locks and decide what to do

10.6 File Locking Example: Java API

- ```java
import java.io.*;
import java.nio.channels.*;
public class LockingExample {
  public static final boolean EXCLUSIVE = false;
  public static final boolean SHARED = true;
  public static void main(String arsg[]) throws IOException {
    FileLock sharedLock = null;
    FileLock exclusiveLock = null;
    try {
      RandomAccessFile raf = new RandomAccessFile("file.txt", "rw");
      // get the channel for the file
      FileChannel ch = raf.getChannel();
      // this locks the first half of the file - exclusive
      exclusiveLock = ch.lock(0, raf.length()/2, EXCLUSIVE);
      /** Now modify the data . . . */
      // release the lock
      exclusiveLock.release();
      // this locks the second half of the file - shared
      sharedLock = ch.lock(raf.length()/2+1, raf.length(), SHARED);
      /** Now read the data . . . */
      // release the lock
      sharedLock.release();
      raf.close();
    } catch (java.io.IOException ioe) {
      System.err.println(ioe);
    } finally {
      if (exclusiveLock != null)
        exclusiveLock.release();
      if (sharedLock != null)
        sharedLock.release();
    }
  }
}
```
```

10.7 File Types: Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

Common File Types and Their Functions

-

10.8 File Structure

- None: Sequence of Words, Bytes
- Simple Record Structure
  - Lines
  - Fixed length
  - Variable length
- Complex Structures
  - Formatted document
  - Relocatable load file
- Can Simulate Last Two with first Method
  - By inserting appropriate control characters
- Who Decides:
  - OS or program

10.9 Access Methods

- Sequential Access

```
read_next();
write_next();
reset();
/* no read after last write(rewrite) */
```

- Direct Access: file is fixed length logical records

```
read(int n);
write(int n, char *data);
/* n = relative block number */
```

- Random Access (Can Access in any Arbitrary Order)
- Relative Block Numbers Allow OS to decide where File should be Placed

10.10 Simulation of Sequential Access on Direct Access File

Sequential Access	Direct Access
`reset();`	`cp=0;`
`read_next();`	`cp++; read(cp);`
`write_next();`	`cp++; write(cp, data);`

Simulation of Sequential Access on Direct Access File

-

10.11 Unix/POSIX Idea: Everything is a File

- Identical Interface for:
  - Files on disk
  - Devices (terminals, printers, etc.)
  - Regular files on disk
  - Networking (sockets)
  - Local inter-process communication (pipes, sockets)
- Based on system calls `open()`, `read()`, `write()`, and `close()`
- Additional: `ioctl()` for custom configuration that doesn't quite fit
- Note that Everything is a File idea was a radical idea when proposed

10.12 Connecting Processes, File Systems, and Users

- Every process has current working directory (CWD)
  - Can be set with system call
  - `int chdir(const char *path); // change CWD`
- Absolute paths ignore CWD
  - `/home/kiarash/file.txt`
- Relative paths are relative to CWD
  - `index.html`, `./index.html`
    - Refers to `index.html` in current working directory
  - `../index.html`
    - Refers to `index.html` in parent directory of current working directory
  - `index.html`, `kiarash/index.html`
    - Refers to `index.html` in the home directory

10.13 I/O and Storage Layers

- Application/Service Layer
- High-Level I/O: Streams (buffered I/O)
- Low-Level I/O: File Descriptors (unbuffered I/O)
- Syscalls: `open()`, `read()`, `write()`, `close()`
- File System Layer: Files, Directories, Indexes
- I/O Driver Layer: Commands and Data Transfers
- I/O Hardware Layer: Disks, Flash, Controllers, DMA

`stdin` (Standard Input)	0	Used by a program to read input
`stdout` (Standard Output)	1	Used by a program to write normal output
`stderr` (Standard Error)	2	Used by a program to write error output

I/O Standard Streams

-

10.14 C High-Level File API: Streams

- Operates on streams: unformatted sequences of bytes (wither text or binary data), with a position:

```c
#include <stdio.h>
FILE *fopen(const char *filename, const char *mode);
int fclose(FILE *fp);
```

Mode Text	Binary	Description
`r`	`rb`	Open existing file for reading
`w`	`wb`	Open file for writing, created if does not exist
`a`	`ab`	Open file for appending, created if does not exist
`r+`	`rb+`	Open existing file for reading and writing
`w+`	`wb+`	Open file for reading and writing, truncated to zero if exists, create otherwise
`a+`	`ab+`	Open file for reading and appending, created if does not exist, Read from beginning, write as append

Modes for `fopen()`

  -

- Open stream represented by pointer to a FILE data structure
  - Error reported by returning a NULL pointer

10.15 C API Standard Streams: stdio.h

- Three predefined streams are opened implicitly when the program is executed
- `FILE *stdin`: normal source of input, can be redirected
- `FILE *stdout`: normal source of output
- `FILE *stderr`: diagnostics and errors
- `stdin`/`stdout` enable composition in Unix
- All can be redirected
  - `cat hello.txt | grep "World!"`
  - cat's `stdout` goes to grep's `stdin`

10.16 C High-Level File API

```c
// character oriented
int fputc(int c, FILE *fp);          // rtn c or EOF on err
int fputs(const char *s, FILE *fp); // rtn > 0 or EOF
int fgetc(FILE *fp);
char *fgets(char *buf, int n, FILE *fp);
```

```c
// block oriented
size_t fread(void *ptr, size_t size_of_elements, size_t number_of_elements, FILE *a_file);
size_t fwrite(const void *ptr, size_t size_of_elements, size_t number_of_elements, FILE *a_file);
// formatted
int fprintf(FILE *restrict stream, const char *restrict format, ...);
int fscanf(FILE *restrict stream, const char *restrict format, ...);
```

10.17 C Streams: Char-by-Char I/O

```c
int main(void)
{
    FILE *input = fopen("input.txt", "r");
    FILE *output = fopen("output.txt", "w");
    int c;
    c = fgetc(input);
    while (c != EOF)
    {
        fputc(output, c);
        c = fgetc(input);
    }
    fclose(input);
    fclose(output);
}
```

10.18 C Streams: Block-by-Block I/O

```c
#define BUFFER_SIZE 1024
int main(void)
{
    FILE *input = fopen("input.txt", "r");
    FILE *output = fopen("output.txt", "w");
    char buffer[BUFFER_SIZE];
    size_t length;
    length = fread(buffer, BUFFER_SIZE, sizeof(char), input);
    while (length > 0)
    {
        fwrite(buffer, length, sizeof(char), output);
        length = fread(buffer, BUFFER_SIZE, sizeof(char), input);
    }
    fclose(input);
    fclose(output);
}
```

10.19 C High-Level File API: Positioning Pointer

```c
int fseek(FILE *stream, long int offset, int whence);
long int ftell(FILE *stream);
void rewind(FILE *stream);
```

- For `fseek()`, the offset is interpreted based on the whence argument (constants in `stdio.h`)
  - `SEEK_SET`: Then offset interpreted from beginning (position 0)
  - `SEEK_END`: Then offset interpreted backwards from end of file
  - `SEEK_CUR`: Then offset interpreted from current position
- Overall preserves high-level abstraction of a uniform stream of objects

10.20 Low-Level File I/O: RAW system-call interface

```c
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
int open (const char *filename, int flags, mode_t mode);
/*
    flags -> Bit vector of:
        - Access modes (Rd, Wr, ...)
        - Open Flags (Create, ...)
        - Operating modes (Appends, ...)
    mode -> Bit vector of Permission Bits:
        - User|Group|Other -> R|W|X
*/
int create (const char *filename, mode_t mode);
int close (int filedes);
```

- Integer return from `open()` is a file descriptor
  - Error indicated by return ¡ 0: the global errno variable set with error (see man pages)
- Operations on file descriptors:
  - Open system call created an open file description entry in system-wide table of open files
  - Open file description object in the kernel represents an instance of an open file
  - Why give user an integer instead of a pointer to the file description in kernel?

10.21 Filesystem

- Filesystem
  - Collection of Files and Directory Structure
- Directory Structure
  - A collection of nodes containing information on all files
  - Both directory structure and files reside on disk

10.22 Building a File System

- File System: Layer of OS that transforms block interface of disks (or other block devices) into Files, Directories, etc.
- Classic OS situation: Take limited hardware interface (array of blocks) and provide a more convenient/useful interface with:
  - Naming: Find file by name, not block numbers
  - Organize file names with directories
  - Organization: Map files to blocks
  - Protection: Enforce access restrictions
  - Reliability: Keep files intact despite crashes, hardware failures, etc.

10.23 User vs. System View of a File

- User's view:
  - Durable Data Structures
- System's view (system call interface):
  - Collection of Bytes (UNIX)
  - Doesn't matter to system what kind of data structures you want to store on disk!
- System's view (inside OS):
  - Collection of blocks (a block is a logical transfer unit, while a sector is the physical transfer unit)
  - Block size $\geq$ sector size; in UNIX, block size is 4KB

10.24 Disk Management

- Basic entities on a disk:
  - File: user-visible group of blocks arranged sequentially in logical space
  - Directory: user-visible index mapping names to files
- Disk is accessed as linear array of sectors
- How to identify a sector?
  - Physical position
    - Sectors is a vector (cylinder, surface, sector)
    - Not used anymore
    - OS/BIOS must deal with bad sectors
  - Logical Block Addressing (LBA)
    - Every sector has integer address
    - Controller translates from address $\rightarrow$ physical position
    - Shields OS from structure of disk

10.25 Disk Structure

- Disk can be Subdivided into Partitions
- Disks or Partitions can be RAID protected against failure
- Disk or Partition can be Used Raw: without a file system, or formatted with a file system
- Partitions also Known as Minidisks, Slices
- Entity Containing File System known as a Volume
- Each Volume Containing File System also tracks that file system's info in Device Directory or Volume Table of Contents
- General-Purpose FS & special-Purpose FS, frequently all within same OS or computer

10.26 What Does File System Need?

- Track free disk blocks
  - Need to know where to put newly written data
- Track which blocks contain data for which files
  - Need to know where to read a file from
- Track files in a directory
  - Find list of file's blocks given its name
- Where do we maintain all of this?
  - Somewhere on disk

10.27 Types of File Systems

- We mostly Focus on General-Purpose FS
- Systems may have several types of FS
  - Some general and some special purpose
- Consider Solaris has:
  - tmpfs: memory-based volatile FS for fast, temporary I/O
  - objfs: interface into kernel memory to get kernel symbols for debugging
  - ctfs: contract file system for managing daemons
  - lofs: loopback file system allows one FS to be accessed in place of another
  - procfs: kernel interface to process structures
  - ufs, zfs: general purpose file systems
- Disk File Systems
  - Ext, ext2/3, FAT, HFS, NTFS, ZFS
- File Systems with Built-in Fault Tolerance
  - BTRFS
- File Systems Optimized for SSDs/Flash
  - JFFS, TrueFFS
- Distributed File Systems
- Distributed Parallel File Systems
- Distributed Parallel Fault-Tolerant File Systems
  - Ceph, Google File System (GFS), Hadoop FS

10.28 Critical Factors in File System Design

- (Hard) Disks Performance !!!
  - Maximize sequential access, minimize seeks
- Open before Read/Write
  - Can perform protection checks and look up where the actual file resource are, in advance
- Size is determined as they are used !!!
  - Can write (or read zeros) to expand the file
  - Start small and grow, need to make room
- Organized into directories
  - What data structure (on disk) for that?
- Need to carefully allocate/free blocks
  - Such that access remains efficient

10.29 Operations Performed on Directory

- Directory
  - Symbol table that translates file names into their directory entries
- Operations on Directory
  - Search for a file
  - Create a file
  - Delete a file
  - List a directory
  - Rename a file
  - Traverse file system

10.30 Directory Organization

- Directory is organized logically to obtain:
  - Efficiency: locating a file quickly
  - Naming: convenient to users
    - Two users can have same name for different files
    - Same file can have several different names
  - Grouping – logical grouping of files by properties (e.g., all Java programs, all games, . . .)

10.31 Single-Level Directory

- A Single Directory for All Users
- Naming Problem
- Grouping Problem

10.32 Two-Level Directory

- Separate Directory for each User
- Path name
- Can have same file name for different user
- Efficient searching
- Pros
  - Solves name-collision problem
  - Creates protections between users
- Cons
  - Sharing a file between two users not allowed
  - No grouping capability

10.33 Tree-Structured Directories

- Absolute or Relative Path Name
- Creating a new file is done in current directory
- Delete a file
  - `rm <file-name>`
- Creating a new subdirectory is done in current directory
  - `mkdir <dir-name>`

10.34 Acyclic-Graph Directories

- Shared Subdirectories and Files
- Two different names (aliasing)
- If dict deletes list → dangling pointer
- Solutions:
  - Backpointers, so we can delete all pointers Variable size records a problem
  - Backpointers using a daisy chain organization
  - Entry-hold-count solution
- New directory entry type
  - Link: another name (pointer) to an existing file
  - Resolve the link: follow pointer to locate the file

10.35 General Graph Directory

- How do We Guarantee No Cycles?
  - Allow only links to file not subdirectories
  - Garbage collection
  - Every time a new link is added use a cycle detection algorithm to determine whether it is OK

10.36 File System Mounting

- A File System must be mounted before it can be accessed
- A Unmounted File System mounted at a mount point

10.37 File-System Structure

- File Structure
  - Logical storage unit
  - Collection of related information
- File System resides on Secondary Storage (disks)
  - Provided user interface to storage, mapping logical to physical
  - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily
- Disk provides in-place rewrite and random access
  - I/O transfers performed in blocks of sectors (e.g., 512B)
- File Control Block (FCB): storage structure consisting of information about a file
- Device Driver controls physical device
- File system organized into layers
  - Application program → Logical file system → Basic file system → I/O control → Devices

10.38 File System Layers

- Device Drivers
  - Manage I/O devices at I/O control layer
  - Given commands like "read drive1, cylinder 72, track 2, sector 10, into memory location 1060" outputs low-level hardware specific commands to hardware controller
- Basic File System
  - Gives command like "retrieve block 123" translates to device driver
  - Also manages memory buffers and caches (allocation, freeing, replacement)
    - Buffers hold data in transit
    - Caches hold frequently used data to enhance performance
- File Organization Module
  - Understands files, logical address, and physical blocks
  - Translates logical block number to physical block number
  - Manages free space, disk allocation
- Logical File System
  - Manages metadata info
  - Translates file name into file number, file handle, location by maintaining file control blocks (inodes in UNIX)
  - Directory management
  - Protection
- Pros
  - Useful for reducing complexity and redundancy
- Cons
  - Adds overhead and can decrease performance
- Main Task:
  - Translates file name into file number, file handle, location by maintaining file control blocks (inodes in UNIX)
  - Logical layers can be implemented by any coding method according to OS designer

10.39 File-System Implementation

- We have system calls at API level, but how do we implement their functions?
  - On-disk and in-memory structures
- Boot Control Block contains info needed by system to boot OS from that volume
  - Needed if volume contains OS, usually first block of volume
- Volume Control Block (superblock, master file table) contains volume details
  - Total number of blocks, number of free blocks, block size, free block pointers or array
- Directory structure organizes the files
  - Names and inode numbers, master file table
- Per-file File Control Block (FCB) contains many details about the file
  - inode number, permissions, size, dates
  - NFTS stores into in master file table using relational DB structures

10.40 In-Memory File System Structures

- Mount Table Storing File System Mounts, Mount Points, File System Types
- Following Figure Illustrates
  - Necessary file system structures provided by OS
  - Plus buffers hold data blocks from secondary storage
  - Open returns a file handle for subsequent use
  - Data from read eventually copied to specified user process memory address

10.41 Partitions and Mounting

- Partition can be a volume containing a file system (cooked) or raw: just a sequence of blocks with no file system
- Boot Block can point to boot volume or boot loader set of blocks that contain enough code to know how to load the kernel from the file system
  - Or a boot management program for multi-os booting
- Root Partition contains OS
  - Other partitions can hold other Oses, other file systems, or be raw
  - Mounted at boot time
  - Other partitions can mount automatically or manually
- At mount time, file system consistency checked
  - Is all metadata correct?
  - If not, fix it, try again
  - If yes, add to mount table, allow access

10.42 Virtual File Systems

- Virtual File Systems (VFS) on Unix provide an object-oriented way of implementing file systems
- VFS allows the same system call interface (the API) to be used for different types of file systems
  - Separates file-system generic operations from implementation details
  - Implementation can be one of many file systems types, or network file system
    - Implements vnodes which hold inodes or network file details
  - Then dispatches operation to appropriate file system implementation routines
- API is to VFS interface, rather than any specific type of file system

10.43 Virtual File System Implementation

- For example, Linux has four object types:
  - inode, file, superblock, dentry
- VFS defines set of operations on the objects that must be implemented
  - Every object has a pointer to a function table
  - Function table has addresses of routines to implement that function on that object

10.44 Directory Implementation

- Linear list of file names with pointer to the data blocks
  - Simple to program
  - Time-consuming to execute
  - Linear search time
  - Could keep ordered alphabetically via linked list or use B+ tree
- Hash Table: linear list with hash data structure
  - Decreases directory search time
  - Collisions: situations where two file names hash to the same location
  - Only good if entries are fixed size, or use chained-overflow method

10.45 Allocation Methods: Contiguous

- An allocation method refers to how disk blocks are allocated for files
- Contiguous Allocation: each file occupies set of contiguous blocks
  - Best performance in most cases
  - Simple: only starting location (block number) and length (number of blocks) are required
  - Problems include finding space for file, knowing file size, external fragmentation, need for compaction off-line (downtime) or on-line

10.46 Extent-Based Systems

- Many newer file systems (i.e., Veritas File System) use a modified contiguous allocation scheme
- Extent-based file systems allocate disk blocks in extents
- An Extent is a contiguous block of disks
  - Extents are allocated for file allocation
  - A file consists of one or more extents

10.47 Allocation Methods: Linked

- Linked allocation: each file a linked list of blocks
  - File ends at nil pointer
  - No external fragmentation
  - Each block contains pointer to next block
  - No compaction, external fragmentation
  - Free space management system called when new block needed
  - Improve efficiency by clustering blocks into groups but increases internal fragmentation
  - Reliability can be a problem
  - Locating a block can take many I/Os and disk seeks
- FAT (File Allocation Table) variation
  - Beginning of volume has table, indexed by block number
  - Much like a linked list, but faster on disk and cacheable
  - New block allocation simple

10.48 Linked Allocation

- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk
- Block to be accessed is the Qth block in the linked chain of blocks representing the file

10.49 Allocation Methods: Indexed

- Indexed allocation
  - Each file has its own index block(s) of pointers to its data blocks
- Logical view
- Need index table
- Random access
- Dynamic access without external fragmentation, but have overhead of index block
- Mapping from logical to physical in a file of maximum size of 256K bytes and block size of 512 bytes. We need only 1 block for index table
- Mapping from logical to physical in a file of unbounded length (block size of 512 words)
- Linked scheme: Link blocks of index table (no limit on size)
- Two-level index (4K blocks could store 1,024 four-byte pointers in outer index → 1048567 data blocks and file size of up to 4GB)

10.50 Performance

- Best method depends on file access type
  - Contiguous great for sequential and random
- Linked good for sequential, not random
- Declare access type at creation → select either contiguous or linked
- Indexed more complex
  - Single block access could require 2 index block reads then data block read
  - Clustering can help improve throughput, reduce CPU overhead
- Adding instructions to the execution path to save one disk I/O is reasonable

10.51 Free-Space Management

- File system maintains free-space list to track available blocks/clusters
- Bit vector or bit map (n blocks)
- Bit map requires extra space
- Easy to get contiguous files

10.52 Linked Free Space List on Disk

- Linked list (free list)
  - Cannot get contiguous space easily
  - No waste of space
  - No need to traverse the entire list (if number free blocks recorded)
- Grouping
  - Modify linked list to store address of next n-1 free blocks in first free block, plus a pointer to next block that contains free-block-pointers (like this one)
- Counting
  - Because space is frequently contiguously used and freed, with contiguous-allocation allocation, extents, or clustering

    - Keep address of first free block and count of following free blocks
    - Free space list then has entries containing addresses and counts
- Space Maps
    - Used in ZFS
    - Consider meta-data I/O on very large file systems
        - Full data structures like bit maps couldn't fit in memory → thousands of I/Os
    - Divides device space into metaslab units and manages metaslabs
        - Given volume can contain hundreds of metaslabs
    - Each metaslab has associated space map
        - Uses counting algorithm
    - But records to log file rather than file system
        - Log of all block activity, in time order, in counting format
    - Metaslab activity → load space map into memory in balanced-tree structure, indexed by offset
        - Replay log into that structure
        - Combine contiguous free blocks into single entry

10.53 Efficiency and Performance

- Efficiency dependent on:
    - Disk allocation and directory algorithms
    - Types of data kept in file's directory entry
    - Pre-allocation or as-needed allocation of metadata structures
    - Fixed-size or varying-size data structures
- Performance
    - Keeping data and metadata close together
    - Buffer cache: separate section of main memory for frequently used blocks
    - Synchronous writes sometimes requested by apps or needed by OS
        - No buffering/caching: writes must hit disk before acknowledgement
        - Asynchronous writes more common, buffer-able, faster
    - Free-behind and read-ahead: techniques to optimize sequential access
    - Reads frequently slower than writes

10.54 Page Cache

- A Page Cache
    - Caches pages rather than disk blocks using virtual memory techniques and addresses
- Memory-Mapped I/O uses a page cache
- Routine I/O through file system uses buffer (disk) cache

10.55 Unified Buffer Cache

- Uses same page cache to cache both memory-mapped pages and ordinary file system I/O to avoid double caching

10.56 Recovery

- Consistency checking: compares data in directory structure with data blocks on disk, and tries to fix inconsistencies
    - Can be slow and sometimes fails
- Use system programs to back up data from disk to another storage device (magnetic tape, other magnetic disk, optical)
- Recover lost file or disk by restoring data from backup

10.57 Log Structured File Systems

- Log structured (or journaling) file systems record each metadata update to the file system as a transaction
- All transactions are written to a log
  - A transaction is considered committed once it is written to the log (sequentially)
  - Sometimes to a separate device or section of disk
  - However, file system may not yet be updated
- Transactions in the log are asynchronously written to the file system structures
  - When the file system structures are modified, the transaction is removed from the log
- If the file system crashes, all remaining transactions in the log must still be performed
- Faster recovery from crash, removes chance of inconsistency of metadata

11 Mass-Storage Systems

11.1 Overview of Mass Storage Structure

- Magnetic Disks provide Bulk of Secondary Storage of Modern Computers
  - Drives rotate at 60 to 250 times per second
  - Transfer rate is rate at which data flow between drive and computer
  - Positioning time (random-access time)
    - Time to move disk arm to desired cylinder (seek time)
    - Time for desired sector to rotate under disk head (rotational latency)
  - Head crash results from disk head making contact with disk surface
- Disks can be Removable
- Drive Attached to Computer via I/O bus
  - Busses vary, including EIDE, ATA, SATA, USB, Fibre Channel, SCSI, SAS, Firewire
  - Host controller in computer uses bus to talk to disk controller built into drive or storage array
- Hard Disks
  - Platters Range from 0.85" to 24" (Historically)
  - Range from 300GB to 16TB per drive
  - Performance
    - Transfer Rate (theoretical): 6 or 12 Gb/sec
    - Effective Transfer Rate (real): 1Gb/sec
    - Seek time from 3ms to 12ms: 9ms common for desktop drives
    - Avg seek time measured or calculated based on 1/3 of tracks
    - Latency based on spindle speed
      - 1 / (RPM / 60) = 60 / RPM
    - Average latency = 0.5 latency

11.2 Hard Disk Performance

- Access Latency = Average access time = average seek time + average rotational latency
- Avg I/O time = avg access time + (amount to transfer / transfer rate) + controller overhead

11.3 Solid-State Disks

- Non-Volatile Memory Used like a Hard Drive
- Can be More Reliable than HDDs
- More Expensive per GB
- Maybe have Shorter Life Span
- Less Capacity
- Much Faster
  - No moving parts → no seek time or rotational latency
- Busses can be too slow → connect directly to PCI for example (e.g., NVMe SSDs)

11.4 Magnetic Tape

- Was Early Secondary-Storage Medium
  - Evolved from open spools to cartridges
- Relatively Permanent and Holds Large Quantities of Data
- Access Time Very Slow
- Random access 1000X slower than HDD
- Mainly Used for Backup, Storage of Infrequently-used data, Transfer Medium between Systems
- Kept in spool and wound or rewound past read-write head
- Once Data under Head, Transfer Rates Comparable to Disk
- 140MB/sec and greater
- 200GB to 30+TB Typical Storage
- Common technologies are LTO-3,4,5 and T10000

11.5 Disk Structure

- Disk Drives are Addressed as Large 1-dim Arrays of Logical Blocks
  - where logical block is smallest unit of transfer
  - Low-level formatting creates logical blocks on physical media
- 1-dim array of logical blocks mapped into sectors of disk sequentially
  - Sector 0: first sector of first track on outermost cylinder
- Mapping Proceeds in Order through
  - Track and then rest of tracks in that cylinder
  - Then through rest of cylinders from outermost to innermost
- Logical to Physical Address should be Easy
  - Except for bad sectors
  - Non-constant number of sectors per track via constant angular velocity

11.6 DAS vs. NAS vs. SAN

- Direct Attached Storage (DAS)
- Network Area Storage (NAS)
- Storage Area Network (SAN)

11.7 Disk Attachment

- Host-Attached Storage accessed through I/O ports talking to I/O busses
- SCSI itself is a bus, up to 16 devices on one cable, SCSI initiator requests operation and SCSI targets perform tasks
  - Each target can have up to 8 logical units (disks attached to device controller)
- FC is High-Speed Serial Architecture
  - Can be switched fabric with 24-bit address space: the basis of storage area networks (SANs) in which many hosts attach to many storage units
- I/O directed to bus ID, device ID, logical unit (LUN)

11.8 Storage Array

- Can just attach disks, or arrays of disks
- Storage Array has controller(s), provides features to attached host(s)
  - Ports to connect hosts to array
  - Memory, controlling software (sometimes NVRAM, etc)
  - A few to thousands of disks
  - RAID, hot spares, hot swap (discussed later)
  - Shared storage → more efficiency
  - Features found in some file systems
    - Snaphots, clones, thin provisioning, replication, deduplication, etc

11.9 Storage Area Network

- Common in Large Storage Environments
- Multiple hosts attached to multiple storage arrays - flexible
- SAN is one or more storage arrays
  - Connected to one or more Fibre Channel switches
- Hosts also attach to switches
- Storage made available via LUN Masking from specific arrays to specific servers
- Easy to add or remove storage, add new host and allocate it storage
  - Over low-latency Fibre Channel fabric
- Why have separate storage networks and communications networks?
  - Consider iSCSI, FCOE

11.10 Network-Attached Storage

- Network-Attached Storage (NAS)
  - Storage made available over a network rather than over a local connection (such as a bus)
  - Remotely attaching to file systems
- NFS and CIFS are common protocols
- Implemented via remote procedure calls (RPCs) between host and storage over typically TCP or UDP on IP network
- iSCSI protocol uses IP network to carry SCSI
  - Remotely attaching to devices (blocks)

11.11 Why to use SAN instead of Local Drives?

- Easier & More Efficient Capacity Management
- High Availability
- High Reliability
- High Throughput
- Low Latency
- But More Expensive

11.12 SSDs: Quick Introduction

- Flash-Based Solid State Drives
  - Non-volatile
  - Consist of NAND flashes

11.13 Applications and Data Accesses

- Data is Highly Skewed
- At Any Given Time, Only a Small Percentage of Data is Active

11.14 Hybrid Storages: Tiering

- Move hot data to high-performance tier
- Move cold data to low-performance tier
- Common tiers (used in enterprise storages)
  - Extreme performance tier: Flash drives
  - Performance tier: SAS drives
  - Capacity tier: NL-SAS drives

11.15 Disk Scheduling

- OS is responsible for using HW Efficiently
  - For disk drives, this means having a fast access time and disk bandwidth
- Minimize Seek Time
- Seek Time is related to Seek Distance
- Disk Bandwidth is Total Number of Bytes Transferred, divided by Total Time between first Request for Service and Completion of Last Transfer
- There are Many Sources of Disk I/O Request
  - OS
  - System processes
  - Users processes
- I/O Request Includes:
  - Input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
  - Optimization algorithms only make sense when a queue exists
- Note that drive controllers have small buffers and can manage a queue of I/O requests (of varying "depth")
- Several algorithms exist to schedule servicing of disk I/O requests
- Analysis is true for one or many platters

11.16 SSTF

- Shortest Seek Time First (SSTF) Scheduling
  - Selects request with minimum seek time from current head position
  - Is a form of SJF scheduling
  - May cause starvation of some requests

11.17 SCAN

- Disk arm starts at one end of disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- Sometimes called Elevator Algorithm
- If requests are uniformly dense, largest density at other end of disk and those wait longest

11.18 C-SCAN

- Provides a more uniform wait time than SCAN
- Head moves from one end of disk to the other, servicing requests as it goes
    - When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats cylinders as a circular list that wraps around from the last cylinder to the first one

11.19 C-LOOK

- LOOK a version of SCAN
- C-LOOK a version of C-SCAN
    - Arm only goes as far as last request in each direction, then reverses direction immediately
    - Without first going all the way to end of disk

11.20 Selecting a Disk-Scheduling Algorithm

- SSTF is common and has a Natural Appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on disk
- Less starvation
- Performance depends on number and types of requests
- Requests for disk service can be influenced by file-allocation method
    - And metadata layout
- Disk-Scheduling algorithm should be written as a separate module of OS, allowing it to be replaced with a different algorithm if necessary
- Either SSTF or LOOK is a reasonable choice for default algorithm
- What about Rotational Latency?
    - Difficult for OS to calculate
- How does disk-based queuing effect OS queue ordering efforts?

11.21 Disk Management

- Low-level Formatting, or Physical Formatting: Dividing a disk into sectors that disk controller can read and write
    - Each sector can hold header information, plus data, plus error correction code (ECC)
    - Usually 512 bytes of data but can be selectable
- To use a disk to hold files, OS still needs to record its own data structures on disk
    - Partition disk into one or more groups of cylinders, each treated as a logical disk
    - Logical formatting or “making a file system”
    - To increase efficiency most file systems group blocks into clusters

        - Disk I/O done in blocks
        - File I/O done in clusters
- Raw Disk Access for apps that want to do their own block management, keep OS out of the way (databases for example)
- Boot Block Initializes System
    - Bootstrap is stored in ROM
    - Bootstrap loader program stored in boot blocks of boot partition
- Methods such as Sector Sparing used to handle bad blocks

11.22 Swap-Space Management

- Swap-Space: Virtual memory uses disk space as an extension of main memory
    - Less common now due to memory capacity increases
- Swap-space can be carved out of normal file system, or, more commonly, it can be in a separate disk partition (raw)
- Swap-Space Management
    - 4.3BSD allocates swap space when process starts; holds text segment (program) and data segment
    - Kernel uses swap maps to track swap-space use

11.23 RAID Structure

- Redundant Array of Inexpensive Disks
    - Multiple disk drives provides reliability via redundancy
- Increases Mean Time To Failure
- Mean Time To Repair
    - Exposure time when another failure could cause data loss
- Mean Time To Data Loss
    - Based on above factors
- Disk Striping uses a Group of Disks as one Storage Unit
- Arranged into Six Different Levels
- Improves Performance and Reliability of Storage System by Storing Redundant Data
    - Mirroring or shadowing (RAID 1) keeps duplicate of each disk
    - Striped mirrors (RAID 1+0) or mirrored stripes (RAID 0+1) provides high performance and high reliability
    - Block interleaved parity (RAID 4, 5, 6) uses much less redundancy
- RAID within a storage array can still fail if the array fails, so automatic replication of data between arrays is common
- Frequently, a small number of hot-spare disks are left unallocated, automatically replacing a failed disk and having data rebuilt onto them

11.24 Other Features

- Regardless of where RAID implemented, other useful features can be added
- Snapshot is a view of file system before a set of changes take place (i.e. at a point in time)
- Replication is automatic duplication of writes between separate sites
  - For redundancy and disaster recovery
  - Can be synchronous or asynchronous
- Hot spare disk is unused, automatically used by RAID production if a disk fails to replace the failed disk and rebuild the RAID set if possible
  - Decreases mean time to repair